

分类号 TP316

学号 GS09062031

UDC

密级 公 开

工程硕士学位论文

面向 IPv6 防火墙的高性能
规则匹配关键技术研究实现

硕士生姓名 江峰

工 程 领 域 软件工程

研 究 方 向 网络安全

指 导 教 师 王晓东 研究员

王勇军 研究员

国防科学技术大学研究生院

二〇一一年十月

Research and Implementation of High Performance Rule Matching Key Technology for IPv6 Firewall

Candidate: Jiang Feng

**Advisor: Prof. Wang Xiao-dong
Prof. Wang Yong-jun**

A thesis

**Submitted in partial fulfillment of the requirements
for the professional degree of Master of Engineering
in Software Engineering
Graduate School of National University of Defense Technology
Changsha, Hunan, P.R.China
October, 2011**

独 创 性 声 明

本人声明所呈交的学位论文是我本人在导师指导下进行的研究工作及取得的研究成果。尽我所知,除了文中特别加以标注和致谢的地方外,论文中不包含其他人已经发表和撰写过的研究成果,也不包含为获得国防科学技术大学或其它教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已 在论文中作了明确的说明并表示谢意。

学位论文题目: 面向IPsec防火墙的高性能规则匹配关键技术研究与实现

学位论文作者签名: 江峰 日期: 2011 年 10 月 21 日

学位论文版权使用授权书

本人完全了解国防科学技术大学有关保留、使用学位论文的规定。本人授权国防科学技术大学可以保留并向国家有关部门或机构送交论文的复印件和电子文档,允许论文被查阅和借阅;可以将学位论文的全部或部分内容编入有关数据库进行检索,可以采用影印、缩印或扫描等复制手段保存、汇编学位论文。

(保密学位论文在解密后适用本授权书。)

学位论文题目: 面向IPsec防火墙的高性能规则匹配关键技术研究与实现

学位论文作者签名: 江峰 日期: 2011 年 10 月 21 日

作者指导教师签名: 王晓东 日期: 2011 年 10 月 21 日

目 录

摘 要	i
Abstract	iii
第一章 概述	1
1.1 课题背景	1
1.2 研究内容	2
1.3 国内外研究现状	2
1.4 论文组织和结构安排	3
第二章 Linux 防火墙架构和处理流程分析	4
2.1 Linux 内核包处理流程	4
2.1.1 Linux 内核包处理总体流程	4
2.1.2 Linux 内核包发送流程分析	4
2.1.3 Linux 内核包接收流程分析	7
2.1.4 网卡收发报文软中断分析	8
2.2 Netfilter 框架	9
2.2.1 Netfilter 架构	9
2.2.2 Linux 内核钩子点	11
2.2.3 hook 的管理	11
2.2.4 sockopt 的管理	13
2.3 Iptables 机制	14
2.3.1 Iptables 整体架构分析架构	14
2.3.2 Iptables 相关数据结构分析	15
2.4 Netfilter/Iptables 内核用户通信机制	20
2.5 本章总结	20
第三章 IPv6 防火墙高性能规则匹配算法 HiPACv6 研究	21
3.1 包分类算法	21
3.2 IPv4 防火墙高速规则匹配算法分析	22
3.2.1 目前高速规则匹配算法概述	22
3.2.2 HiPAC 算法	26
3.2.3 选择 HiPAC 算法的原因	28
3.2.4 IPv6 下 HiPAC 算法的性能瓶颈	28
3.3 IPv6 高性能规则匹配算法 HiPACv6	29

3.3.1 特征分析.....	29
3.3.2 IPv6 高性能规则匹配算法基本思想	29
3.3.3 位选取法.....	30
3.3.4 包的匹配流程.....	31
3.3.5 HiPACv6 算法实例分析	31
3.4 本章总结.....	35
第四章 系统实现与关键技术分析	36
4.1 Linux 内核编程简介	36
4.1.1 内核编程与用户态编程比较.....	36
4.1.2 进程上下文与中断上下文区别.....	36
4.1.3 内核编程同步机制.....	37
4.1.4 内核与用户态通信机制.....	37
4.1.5 内核内存管理函数.....	38
4.1.6 内核开发调试.....	38
4.2 HiPACv6 算法的 Linux 内核实现	40
4.2.1 系统总体结构.....	40
4.2.2 用户态部分实现.....	40
4.2.3 内核部分实现.....	41
4.2.4 内核接收命令接口.....	46
4.3 关键技术及难点分析.....	47
4.3.1 HiPACv6 对 UDP 分片包的处理	47
4.3.2 HiPACv6 对加密报文的处理	47
4.3.3 位选取法难点分析.....	48
4.4 本章总结.....	49
第五章 系统测试及结果分析	50
5.1 IPv6 防火墙原型系统配置	50
5.2 测试方案设计.....	50
5.3 防火墙算法的功能测试.....	51
5.4 性能分析.....	52
5.5 本章总结.....	52
第六章 总结与展望	53
6.1 总结.....	53
6.2 展望.....	53

致 谢.....	55
参考文献	56
作者在学期间取得的学术成果	58

表 目 录

表 3.1	匹配域的分块	22
表 3.2	规则的匹配范围和目标	23
表 3.3	规则和规则范围	24
表 3.4	规则的元组空间	25
表 3.5	IPv6 规则匹配范围	32
表 3.6	规则分配到的规则集	33
表 3.7	规则源 IP 地址与目的 IP 地址串接	33
表 3.8	比特位提取	33
表 3.9	比特位 0 和 1 个数差的绝对值	34
表 3.10	规则的重新分配	34
表 3.11	包各个匹配域的值	35
表 4.1	IPv6 扩展头	48
表 5.1	原型系统配置	50
表 5.2	Ip6tables 与 HiPACv6 防火墙吞吐率的测试结果(单位 Mb/s)	52

图 目 录

图 2.1	Linux 内核包处理总体流程.....	5
图 2.2	Netfilter 框架.....	10
图 2.3	Linux 内核钩子点.....	11
图 2.4	hook 数据结构和 hook 链结构	12
图 2.5	钩子点处理流程图	13
图 2.6	hook 的添加	13
图 2.7	新增 hook 链	13
图 2.8	nf_sock_opt 结构	14
图 2.9	sockopt 添加.....	14
图 2.10	iptables 框架.....	16
图 2.11	内核中存放安全规则数据结构	18
图 2.12	内核中处理安全规则方法的结构	19
图 2.13	用户层存放安全规则处理方法的结构	19
图 3.1	规则集对 $B(v)$ 域的划分	24
图 3.2	HiCuts 树的构造	25
图 3.3	TCAM 算法.....	26
图 3.4	HiPAC 算法以及报文查找过程	28
图 3.5	HiPACv6 算法结构图	30
图 3.6	规则的插入	34
图 4.1	系统组成以及模块之间的关系	40
图 4.2	IPv6 头结构.....	47
图 5.1	IPv6 防火墙原型系统测试环境.....	50
图 5.2	HiPACv6 与 Ip6tables 性能比较.....	52

摘 要

随着 Internet 技术的发展,网络安全问题也成为了人们日益关注的焦点。作为内网和外网之间进行安全性访问的桥梁,防火墙在互联网时代有着不可替代的作用。同时,由于 IPv4 地址空间的不足以及 IPv4 暴露出的许多缺点,使得互联网技术体制将由 IPv4 向 IPv6 的过渡和转变成为必然。因此,面向 IPv6 的高性能防火墙技术也成为了网络安全领域的研究重点。本文以最终能在 Linux 内核中实现高性能的 IPv6 防火墙为目标,对包处理过程中的快速规则匹配算法和实现关键技术进行了深入研究,主要完成了以下工作:

首先,对 Linux 防火墙的架构及其处理流程进行了详尽分析,重点对内核包处理的收发流程、Netfilter 框架中的 Hook 管理以及 IPtables 机制中的安全规则数据结构进行了整理和阐述,通过实际系统的调测试,掌握了关键实现细节,为下一步实现完整的 IPv6 防火墙奠定了良好的实现基础;

其次,对防火墙包分类算法的相关研究现状进行了深入分析。其中, Linux 内核中基于顺序查找算法实现规则匹配的方式,在规则数增加时,尤其是企业级防火墙和核心路由器的应用环境中,将导致其性能急剧下降。比较典型的一种优化算法是基于范围定位的 HiPAC(High Performance Packet Classification for Netfilter)算法。通过对每个匹配进行范围划分,构建一个 B+树,把规则的匹配问题转变为范围的定位问题。当规则数增大时,其性能要远远胜过于基于顺序查找的规则匹配算法,并且查找速度几乎不受规则的增加影响。但是,HiPAC 算法内存空间消耗太大,其空间复杂度基于随着规则的增长呈指数增长的趋势。

在分析了现有 IPv4 算法之后,对 IPv6 防火墙的特征进行了分析,发现基于 IPv4 的高性能规则匹配算法并不适用于 IPv6,其主要原因是 IPv6 的地址空间比 IPv4 要大得多,这将导致地址范围的急剧增大,使得空间复杂度很高,进而提出了一种适用于 IPv6 的高性能规则匹配算法 HiPACv6。该算法综合了范围定位匹配和线性匹配各自的优点,折中了算法的时间复杂度和空间复杂度。首先根据协议域和目的端口域特征,将整个规则集分成 20 个子规则集,然后再对源 IPv6 地址和目的 IPv6 地址的组合进行位选取,构建二叉查找树。最后分别把 20 个子集分成若干小的规则集合。当报文匹配到某个小的规则集合时,再线性匹配小规则集中的规则。这样防火墙性能比较于 Linux 内核中的 Ip6tables 实现有了极大的提高,并且有效地控制了内存的使用。

最后,我们在 Linux 内核中实现了基于 HiPACv6 算法的 IPv6 防火墙原型系统,并对其关键实现技术进行了分析,包括 HiPACv6 对 UDP 分片包的处理,HiPACv6 对加密报文的处理和位选取法。通过实验验证,在大规则集(3000 条规则以上)下,

两台千兆网卡主机之间的通信带宽仍可达到 700Mbps, 远远高于 Ip6tables 100M 左右的带宽。表明基于 HiPACv6 算法实现的 IPv6 防火墙具有较高的处理性能, 可以满足企业级防火墙的应用需求。

关键词: Linux; 防火墙; IPv6; 规则匹配; 包过滤; 高性能

Abstract

With the development of Internet, People pay more and more attention to network security. Firewall, which is a bridge of the interaction between internal network and external network, plays an important role in network times. However, due to the sufficient IP address and so much weakness of IPv4 network system, it is destined for network system to transform from IPv4 to IPv6. Thus, a high performance rule matching algorithm based on IPv6 has become a key research point in network security field. Aiming to being implemented high performance IPv6 firewall in linux kernel, this paper details in analysing high speed rule matching in packet classification and the key point in implementing this algorithm. My main working is as follows:

Firstly, the paper details introduction of the architecture of linux firewall and packet flow. Through elaborating packet processing flow of linux kernel, management of hook in Netfilter, data structure of the rule in Iptables and debugging on real system, we can have a detailed knowledge on linux firewall, which is necessary for implementing an intact linux IPv6 firewall in the next step.

Secondly, the paper has a in-depth analysis on the current research status of packet classification of firewall. As is known to all, linux implemented firewall based on linear searching algorithm. With the increasing of the number of rules, especially in the scenes of enterprise firewall or a core router, the performance of this algorithm will dramatically decrease. Even so, there are lots of optimized algorithms, one of which is the HiPAC(High Performance Packet Classification for Netfilter), based on range location. Through splitting a match range into unempty parts, HiPAC algorithm converted rule matching problem into range location problem. Its performance show a greater advantage over algorithms based on linear searching in the case of large number of rules. Besides this, the search performance of HiPAC have nothing to do with increasing of the number of rules. However, HiPAC will consume lots of memory, and its space complexity will grow exponentially with the increase of the number of rules.

Having analyzed existing IPv4 firewall packet classification algorithm, We analyzed the characteristic of IPv6 firewall, finding that those high performance rule matching algorithm based on IPv4 are not suitable for IPv6. The main cause of this situation is that the range of IPv6 address is much bigger than the range of IPv4 address, which will dramatically augment IP address range, consequently augment space complexity. Considering factors referred above, we come up with a high performance rule matching algorithm suitable for IPv6, named HiPACv6. This algorithm colligated the advantage of range location matching algorithm and the advantage of linear searching algorithm, compromised time complexity and space complexity. By classifying the whole rules into 20 parts according to protocol field and destination port

field, this algorithm then select bits from connection between source IPv6 address and destination IPv6 address, constructing binary trees. And finally these 20 rule sets are splitted into umpty small rule sets. When a packet matches one of these rule sets, we then exploit linear searching algorithm to match all rules in this small rule set. Through these policies, we gain a high boost to IPv6 firewall performance. As well as this, the utilizing of memory is under control.

Eventually, we implemented HiPACv6 in linux kernel. And we analyzed the key points of this algorithm, including processing of UDP fragments, processing of encrypted packets and Bit Selection. The result of experiment shows that the communication bandwidth between two hosts equipped with 1000M-NIC can reach 700Mbps under large number of rules, while Ip6tables can only reach 100M bandwidth under the same condition. As a IPv6 firewall, we can see HiPACv6 has a good performance. It can meet the demands of enterprise firewall.

Key Words: Linux, Firewall, IPv6, Rule Matching, Packet Classification, High Performance

第一章 概述

随着网络技术的快速发展,网络的安全性也越来越受到人们的重视,防火墙作为实现内网与外网之间进行安全性访问的桥梁,在网络时代有着不可替代的作用。Linux 作为一种开源的操作系统,在企业级市场扮演者重要的角色。Linux 防火墙的实现是基于 Netfilter/Iptables,它采用线性匹配算法来实现规则的匹配。

不仅网络安全受到关注,网络吞吐率同样是人们最关心的话题之一。这就要求防火墙必须有更快的规则匹配算法,才能保证在大规则集下网络吞吐率符合人们的要求。

1.1 课题背景

防火墙^[1],作为实现内网与外网之间进行安全性访问的桥梁,在网络时代有着不可替代的作用。从实现的方法来划分,防火墙可以分成软件防火墙和硬件防火墙。所谓的软件防火墙,就是用软件的方法实现的防火墙,它的主要特点是实现成本较低,适合于对性能要求不是很高的场合。而硬件防火墙是由硬件来实现算法的防火墙,这类防火墙的实现成本较高,通常适合对性能要求很高的场合。从技术层面上来划分,防火墙主要可以分成网络层防火墙和应用层防火墙。网络层防火墙指的是在 Linux 内核 TCP/IP 协议栈的网络层用规则对进出的报文进行过滤,主要通过源端口、目的端口、协议号、源 IP 地址、目的 IP 地址 5 个匹配域。而应用层防火墙也称代理防火墙,主要是在应用层对数据报文进行过滤。应用层防火墙可以拦截某应用程序所有的封包。

防火墙具有以下基本特性:

- (1) 内部网络和外部网络之间的所有网络数据流都必须经过防火墙;
- (2) 只有符合安全策略的数据流才能通过防火墙;
- (3) 防火墙自身应具有非常强的抗攻击免疫力;

IPv4 时代针对防火墙的规则匹配速度有很多优化的算法,使得高性能规则匹配防火墙得以实现。但是随着网络技术的发展,原有的 IPv4 体系不能满足人们对 IP 地址空间的需要,同时 IPv4 还暴露出来许多缺点,比如骨干路由器中路由表过于庞大等问题,这使得 IPv6^[2]取代 IPv4 成为必然。IPv6 的主要优势在于:扩大地址空间、提高网络的整体吞吐量、改善服务质量、安全性有更好的保证、支持即插即用和移动性、更好的实现多播功能。随着 IPv6 网络的商业化推广,一种快速 IPv6 包分类^[3]算法的研究已经迫在眉睫。Linux 内核本身自带的防火墙是基于顺序查找算法实现的规则匹配。这种算法的缺点就是当规则数增大时,防火墙的性能急剧下降,从而导致网络吞吐率迅速降低。这显然不是人们所希望的。然而,

IPv4 下研究的那些优化算法并不能直接适用于 IPv6 网络，因为 IPv6 比 IPv4 大的多，IP 地址的表示不能用一个简单的数据类型来表示。除此之外，IPv6 和 IPv4 还有很多不同之处。所以研究一种适用于 IPv6 网络的防火墙成为了当今的一个热门课题。

1.2 研究内容及创新点

本文主要研究 IPv6 下高性能规则匹配算法。多维包分类问题一直是路由器相关技术的瓶颈所在，但是它同时又是网络增值业务诸如 QoS、ACL^[4]、区分服务等的基础。包分类算法直接影响着这些业务的好坏。IPv4 环境下，IP 地址的长度为 32bit，相比于 IPv6 地址的 128bit 相差甚远。而且 128bit 的数据也不能用一种简单的数据类型来表示。所以在 IPv6 下选择一种恰当的方法来对 IP 地址的处理成为首先要考虑的问题。

本文将从 Linux 下防火墙的架构分析开始，通过介绍 IPv4 下集中高速规则匹配算法，重点分析 IPv4 下高性能防火墙 HiPAC^[5]算法。分析了 HiPAC 在 IPv6 下的不足之处。通过对以上的分析，并且结合两者各自的优点，提出了一种适用于 IPv6 的高性能规则匹配算法，并在 Linux 内核下得以实现。最后对改进算法进行测试分析。

本文主要有以下创新点：

- (1) 根据 IPv6 地址特征，利用位选取法对规则进行分类。通过对源 IP 地址和目的 IP 地址的组合进行位选取，构建规则的二叉查找树；
- (2) 结合了范围匹配算法和线性规则匹配算法各自的优点，首先根据包的协议号、目的端口号、源 IP 地址、目的 IP 地址判断属于哪个分类子集，然后再线性匹配分类子集中的规则。使得算法的时间复杂度和空间复杂度有了较好的平衡。

1.3 国内外研究现状

防火墙的研究一直备受国内外相关研究人员的关注，特别是在 IPv4 时代，大量针对于 IPv4 防火墙的优化算法已经提出。这其中比较典型的几个算法有 HiPAC、HiCuts^[6]、TSS^[7]、RFC^[8]、BV^[9]等。有的算法空间复杂度比较高，这就要在设计时对时间复杂度和空间复杂度进行折中。IPv4 包分类算法的研究已经相当成熟，然而随着互联网从 IPv4 体系结构向 IPv6 的过度，原有的防火墙优化算法并不能直接适用于 IPv6 网络。这就需要人们通过修改使其适用于 IPv6 网络环境。

一方面，由于 IPv6 的商业化进展一直比较缓慢，因此投入到 IPv6 包分类算法

的研究人员也不是很多，并且由于时间不是很长，所以效果不是很明显。

另一方面，IPv4 环境下包分类算法有一套自己的规则和包头生成工具 ClassBench^[10]，这使得研究人员比较方便的测试算法，而不用自己再去设计规则集和包头。然而 IPv6 下研究的不是很多，所以没有类似这样的一套 ClassBench。这使得 IPv6 包分类算法的研究更为复杂。

对具体研究情况的分析拟在后续章节中介绍。

1.4 论文组织和结构安排

本文共七章，各个章节具体结构如下：

第一章：绪论部分，主要讲述了防火墙相关背景、本文的研究内容及创新点、国内外研究现状和论文组织和结构安排。

第二章：首先分析了 Linux 内核包经过协议栈的处理流程，重点突出了面向防火墙包过滤的包处理流程分析。接着分析了 Linux 内核防火墙模块 Netfilter/Iptables，并且分别对 Netfilter 和 Iptables 框架进行了详细的分析。

第三章：首先对 IPv4 下几种经典的防火墙高速规则匹配算法进行分析，重点对 HiPAC 算法进行分析，突出了 IPv6 下 HiPAC 算法的不足之处。提出了适用于 IPv6 环境下的防火墙高速规则匹配算法 HiPACv6。

第四章：首先介绍了 linux 内核编程的基本方法以及需要注意的地方，然而详细的说明了 HiPACv6 算法实现 Linux 中 IPv6 防火墙的主要过程，并列举出相关关键代码。最后分析了算法在 Linux 内核实现的关键技术难点。

第五章：首先介绍了基于 HiPACv6 算法的 IPv6 防火墙原型系统测试方案，然后利用该测试方案对算法进行功能测试和性能测试，最后分析测试结果。

第六章：对本文的工作进行了总结，同时指出了系统有待改进的地方。

第二章 Linux 防火墙架构和处理流程分析

为了最终基于 Linux 内核实现 IPv6 防火墙，我们必须分析和掌握 Linux 现有防火墙架构和处理流程。Linux 内核基于 Netfilter/Iptables 实现防火墙过滤功能，其实现与复杂的内核 IP 协议栈包处理流程紧密相关，因此，必须对包在 Linux 内核协议栈 IP 层的处理流程有一个清晰的了解。本章以 Linux-2.6.18 内核版本为例，通过代码分析与调试相结合的方法，首先分析 Linux 内核协议栈 IP 层的包处理流程，重点突出包经过 Netfilter 的处理，使得我们对 Linux 内核防火墙包过滤原理有一个直观的理解。然后再深入阐明 Netfilter/Iptables 框架中的技术细节，从微观上阐明 Linux 内核防火墙的实现。

2.1 Linux 内核包处理流程

2.1.1 Linux 内核包处理总体流程

包在 Linux 内核协议栈 IP 层的处理包含两个部分，分别是接收部分和发送部分。接收部分流程如图 2.1 左半部分所示，发送部分流程如图 2.1 右半部分所示。Netfilter^[11]由注册在 Linux 内核协议栈 IP 层的五个钩子点组成。发送包、接收包、转发包分别经过 Netfilter 上不同的钩子点。下面我们重点分析包在 IP 层的处理，特别是在 Netfilter 上的处理。

2.1.2 Linux 内核包发送流程分析

Linux 内核协议栈发送流程在 IP 层的处理如图 2.1 左半部分所示，当应用层调用 socket 系统调用时，例如 socket(AF_INET, SOCK_STREAM, 0)，会指定 L4 层(传输层)使用的协议类型，其中包括 SOCK_STREAM、SOCK_DGRAM、SOCK_RAW 等。如果第四层指定为 SOCK_STREAM 时，表示应用层将使用 Linux 内核协议栈提供的 TCP 协议来通信；如果第四层指定为 SOCK_DGRAM 时，表示应用层将使用 Linux 内核协议栈提供的 UDP 协议来通信；当第四层指定为 SOCK_RAW 时，表示应用层将自行指定双方通信机制，但是这种通信机制是建立的 IP 层基础上的，尽管如此，我们也可以通过 setsockopt 系统调用来设置 IP_HDRINCL 选项跳过 Linux 内核协议栈的 IP 层，让应用层发送的包直接通过链路层在到物理层发送出去。必须注意的是，当我们使用 TCP 或者是 UDP 时，都是对应于一种特定的协议类型，所以 socket 系统调用的协议域可以用 0 来代替，但是当我们用 RAW_SOCKET 时，因为 RAW_SOCKET 可以指定自己的通信机制，

所以每一种通信机制都要设定为一个协议类型，也就是 socket 系统调用中的协议域。所以说当用 socket 系统调用创建两种 raw socket 时，例如 socket(AF_INET, SOCK_RAW, 28)，socket(AF_INET, SOCK_RAW, 32)，相当于定义了两种通信协议类型。

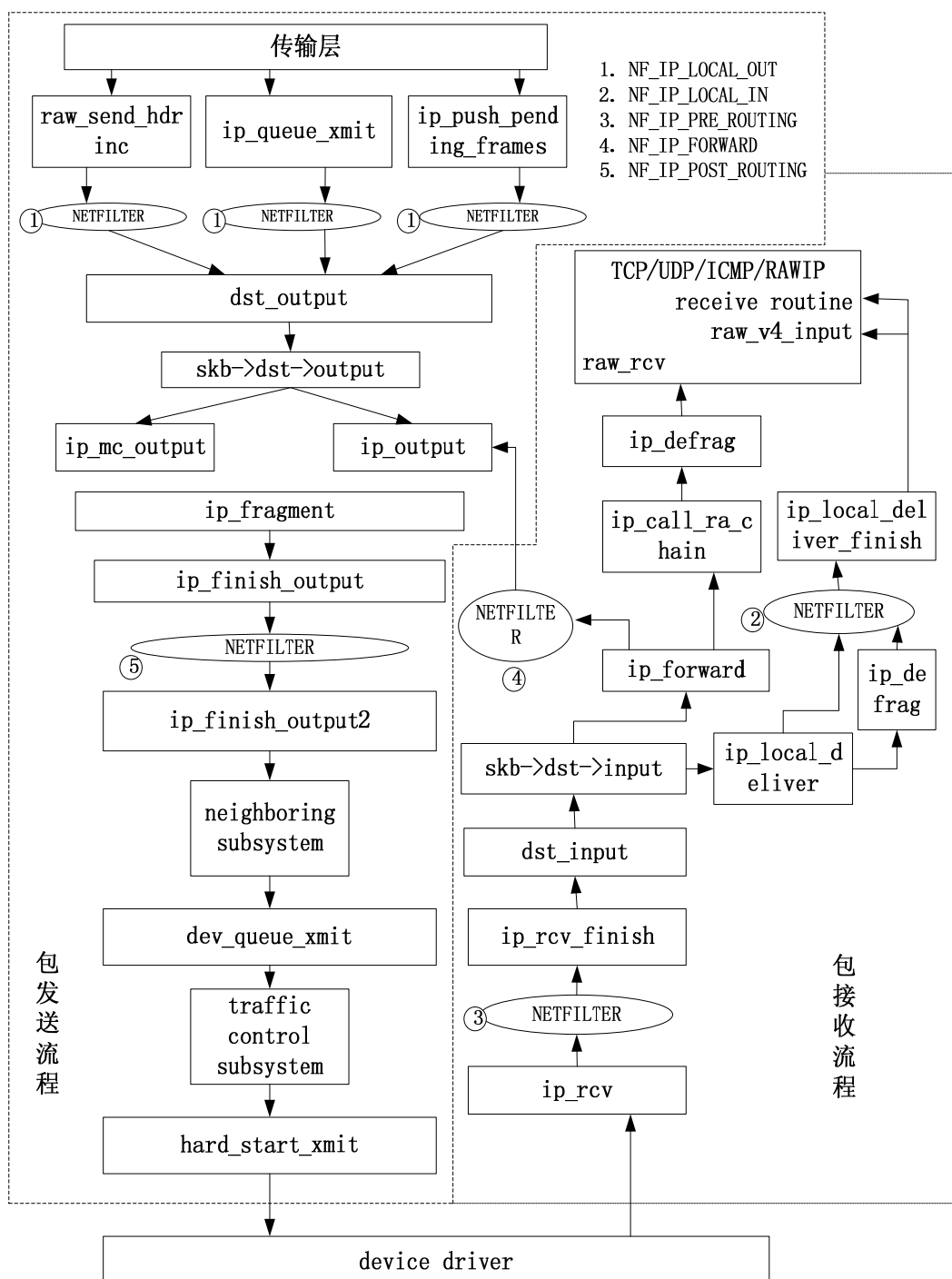


图 2.1 Linux 内核包处理总体流程

当报文在 L4 层处理完成之后，TCP、UDP、RAW_SOCKET 通过不同的接口函数提交给 IP 层处理。对于 TCP 或者是 SCTP 来说，调用的是 ip_queue_xmit 函

数, 由于 TCP 或者 SCTP 协议对分片包的处理都是在 L4 层完成的, 所以留给 IP 层的只是少量的工作。总的来说, ip_queue_xmit 函数的任务主要包括路由的设置和 IP 头的建立; UDP 调用的是 ip_push_pending_frames 函数, 该函数一般与 ip_append_data 或者是 ip_append_page 协同工作, ip_append_data 和 ip_append_page 主要是对分片包进行内存的分配。当分配好适当的内存后, 当包继续往下传的时候, 只要简单的改变指针, 填充 IP 头和 MAC 头; RAW_SOCKET 调用的是 raw_send_hdrinc 函数, 与 UDP 协议相类似, 用 RAW_SOCKET 发包同样是把大部分的分片工作移交给 IP 层处理。与 TCP 处理不相同的是, UDP 和 RAW_SOCKET 将路由的查找在 L4 层完成, 所以当调用 ip_push_pending_frames 函数或者 raw_send_hdrinc 函数时, 只是添加 IP 头和相应的对分片的处理。

当 IP 头添加完成之后, 该包需要经过一个 NF_IP_LOCAL_OUT 钩子点^[12]的处理, Linux 钩子点的设置主要是针对 Linux 防火墙的实现相关, 一般用来禁止某些特定 IP 地址、特定协议、特定端口号、特定服务的报文通过。钩子点处理完成之后, 根据前面选择的路由, 判断发送的报文是单播报文还是多播报文, 调用 dst_output 选择相应的包输出函数, 当发送的包是单播包时, 调用的是 ip_output 函数, 当发送的是多播包时, 调用的是 ip_mc_output 函数。ip_output 和 ip_mc_output 函数首先通过 Netfilter 上的 NF_IP_POST_ROUTING 钩子点, 如果包是允许通过的, 则继续判断包是否需要分片处理, 如果需要, 调用 ip_fragment 函数对包进行分片处理, 最后调用 ip_finish_output2 函数, ip_finish_output2 函数通过调用邻居子系统把包交给 MAC 层处理。

当报文在 Linux 内核协议栈的处理完成之后, 需要经过网卡设备来发送, 内核缓冲队列中的报文到驱动缓冲队列中的传递过程主要是通过 dev_queue_xmit 函数来完成, 这个函数是执行发送设备驱动的接口。dev_queue_xmit 函数首先检查这一帧是否为分片报文并且检查网卡设备是否支持 scatter/gather 功能, 如果是分片帧, 并且网卡不支持 scatter/gather 功能, 则尝试把包线性化(将分布在多个内存区域的数据段合并到单个数据区), 如果线性化不成功, 则丢弃此帧; 其次确认 L4 层校验和已经计算, 如果 L4 层校验和没有计算, 并且网卡不支持硬件校验和时, 在这个完成 L4 层校验和的计算, 如果计算校验和失败, 释放掉这个报文; 最后选择要发送的包, 并且调用 hard_start_xmit 函数发送出去。

由此可知, Linux 内核发送的包在 IP 层处理需要经过两个钩子点。包在路由前经过 NF_IP_LOCAL_OUT 钩子点的处理, 然后路由后的包再经过 NF_IP_POST_ROUTING 钩子点进行处理。本机进程发送出去的包都要通过 NF_IP_LOCAL_OUT 钩子点, 而所有即将通过网络设备发送出去的包都要通过 NF_IP_POST_ROUTING 钩子点。

2.1.3 Linux 内核包接收流程分析

从网卡接收到的报文经过设备驱动程序的处理之后，最终提交到设备驱动与内核的代码的接口函数 `netif_rx`，这个函数的作用等同于报文发送时候 `dev_queue_xmit` 函数。当有新的输入帧需要处理时，网卡设备驱动程序就会调用 `netif_rx` 函数，它的作用就是调度软中断^[13]出列报文并且处理这个报文。`netif_rx` 首先初始化 `sk_buff` 数据结构中相应的域，然后把接收到的帧存储在 CPU 私有输入队列中，通过触发 `NET_RX_SOFTIRQ` 软中断来通知内核接收帧，最后更新拥塞等级统计数据。

当设备驱动层处理完成之后，最终由 `netif_receive_skb` 函数根据 MAC 头的协议号，调用相应的处理函数，例如当协议值为 `0x8000` 时，对应的是 IP 处理函数是 `ip_rcv`。于是，包就开始进入 Linux 内核协议栈代码的处理。首先 `ip_rcv` 函数对包的完整性、正确性进行检查，确认包没有错误时交给 `NF_IP_PRE_ROUTING` 钩子点进行处理，当包被允许接收时，再调用 `ip_rcv_finish` 函数完成包的接收工作，`ip_rcv_finish` 函数首先通过 `ip_route_input` 函数查找路由表，判断该报文是否是发往本地的报文，由此向 `skb->dst->input` 虚函数中注册相应的报文输入函数，如果是发往本地的报文，则 `skb->dst->input` 函数为 `ip_local_deliver`，如果不是发往本地的报文则 `skb->dst->input` 函数为 `ip_forward`。然后由 `ip_rcv_finish` 函数调用前面注册的函数。

针对发往本地的包，通过调用 `ip_local_deliver` 函数，协议栈首先检测报文是否为分片报文，如果是分片报文则首先重组分片报文，最后在交给 `NF_IP_LOCAL_IN` 钩子点进行处理，如果不是分片报文，则直接交给 `NF_IP_LOCAL_IN` 钩子点进行处理。当钩子点处理完成之后，如果包被允许接收，则交给 `ip_local_deliver_finish` 函数进行处理。`ip_local_deliver_finish` 函数把从 L3 层提交到 L4 层，它的主要任务就是根据 IP 头中的 `protocol` 域的值找到相应的协议处理函数，并且把包提交给这个函数。L4 层相应协议对应的处理函数为：ICMP 对应于 `icmp_rcv` 函数，UDP 对应于 `udp_rcv` 函数，TCP 函数对应于 `tcp_v4_rcv` 函数。协议和相应协议处理函数的注册是通过 `inet_add_protocol` 函数来实现的。注册的协议对应的结构体的定义为：

```
struct net_protocol {
    int (*handler)(struct sk_buff *skb); //L4 层接收处理函数
    void (*err_handler)(struct sk_buff *skb, u32 info); // L4 层出错接收处理函数
    int (*gso_send_check)(struct sk_buff *skb);
    struct sk_buff* (*gso_segment)(struct sk_buff *skb, int features);
    int no_policy;
};
```

如果不是发往本地的包，则需要把这个包转发出去，包的转发主要涉及到两个函数，一个是 `ip_forward` 函数，一个是 `ip_forward_finish` 函数，`ip_forward_finish` 函数的调用是在 `ip_forward` 函数的最后，前提是 `ip_forward` 调用 `NF_HOOK` 函数提交给 `NF_IP_FORWARD` 钩子点处理时，返回的裁判值是接受的 (`NF_ACCEPT`)。转发一个包有下面五个步骤组成：

- (1) 处理 IP 选项；
- (2) 根据 IP 头相应的域值，确保这个包可以被转发出去；
- (3) 减少 IP 头中的 TTL 值，如果 TTL 只变成 0 了，直接丢弃此包；
- (4) 如果需要的话，组合分段包，例如 Netfilter 要对包进行处理，需要整个包的信息；
- (5) 把包发送到网卡输出设备。

由此可知，包的 IP 层的接收要经过 `NF_IP_PRE_ROUTING` 钩子点，然后根据目的 IP 地址判断包是否自发往本机的包，如果是，则再经过 `NF_IP_LOCAL_IN` 钩子点的处理，如果不是，则经过 `NF_IP_FORWARD` 钩子点的处理。同时可以看出，经过本机转发的包需要一次经过 `NF_IP_PRE_ROUTING`、`NF_IP_FORWARD`、`NF_IP_POST_ROUTING` 钩子点。

2.1.4 网卡收发报文软中断分析

2.1.4.1 网卡接收报文软中断分析

每个 CPU 都有自己管理输入输出报文的结构，这个结构就是 `softnet_data` 结构体，下面是该结构体的定义：

```
struct softnet_data {
    int  throttle;
    int  cng_level;
    int  avg_blog;
    /* 输入的包队列 */
    struct sk_buff_head input_pkt_queue;
    struct list_head poll_list;
    /* 正在发送包的网络设备链表 */
    struct net_device *output_queue;
    /* 完成队列，每个元素都指向已经发送完成的缓冲区 */
    struct sk_buff *completion_queue;
    struct net_device backlog_dev;
};
```

这个结构体既包含用来发送包的域又包含用来接收包的域，接收到的包进入 `input_pkt_queue` 队列中，出去的包被存入另外一个队列中。

Linux 设备驱动通知内核接收一个新帧的方法有两种，第一种是通过函数

netif_rx 函数的调用,大多数 linux 设备驱动采用这种方法;另外一种是通过 NAPI^[14] 机制。NAPI 机制结合了中断方式和轮询方式^[15]的优点,使得在高负载网络下网络吞吐率与原来中断驱动的方法有了极大的改善。NAPI 大概思想如下:当内核还没有完成对上一个帧处理的时候接收到了一个新帧,这时设备没有必要生成新的中断,关闭设备中断继续处理,当队列为空时,重新开启中断。这样就可以同时利用中断和轮询的优点。一方面帧的接收是异步的,采用中断方式;另一方面如果内核知道设备输入队列中还有没处理的输入帧时,就没有必要在处理中断上浪费时间,采用简单的轮询就可以了。注意,在轮询链表中的设备是关闭了中断的。接收软中断处理函数为 net_rx_action,在基于中断驱动的方式下,每接收到一帧就会产生一次软中断,软中断在调用相应的中断处理函数 net_rx_action,这样当网络流量很高时,就会使得 CPU 疲于中断上下文的切换。而在 NAPI 方式下,net_rx_action 就会遍历处于轮询状态的网卡设备,然后调用每个设备所对应的 poll 虚函数来处理接收到的帧。由于对链表中轮询设备的访问时时间片轮回的,并且在一个调用 poll 方法中有一个最大处理帧个数。如果在给定的时间片内不能处理完队列中所有的输入报文,则它必须等待下次轮询访问。

2.1.4.2 网卡发送报文软中断分析

网卡发送对应于软中断 NET_TX_SOFTIRQ,它所对应的软中断处理函数是 net_tx_action,由设备驱动触发,设备驱动在以下两种情况下会触发软中断 net_tx_action 的执行。

- (1) 当某一个设备可以发送帧时,可以通过调用 netif_wake_queue 函数来触发中断;
- (2) 当一次传输成功并且设备驱动通知该次传输所用到的内存空间可以释放时,通过函数 dev_kfree_skb_irq 可以触发发送软中断 net_tx_action 的执行。

第一种情况主要是用来当满足某些条件,例如设备有了足够的存储空间时,启动软中断继续发送帧。第二种情况主要是用来释放已经发送帧的缓冲空间,我们知道运行在中断上下文的设备驱动程序代码,必须处理的越快越好。然而释放一个缓冲空间需要耗费一定的时间,所以存储空间的释放就被推迟到软中断中,在设备驱动程序中断只是将指向要被释放的存储空间的指针添加到 completion_queue 链表中,真正的释放工作通过软中断函数 net_tx_action 来执行。

2.2 Netfilter 框架

2.2.1 Netfilter 架构

Netfilter 是构成 Linux 防火墙的核心,主要包含防火墙功能实现模块和规则的

修改模块。其中防火墙的功能实现模块主要由 `nf_hook_ops` 结构体的注册实现，规则修改模块主要有 `nf_sockopt_ops` 结构体的注册实现。下面我们将对 Netfilter 框架进行详细的介绍。

如图 2.2 所示，Linux 内核中 Netfilter 架构主要包含以下几个模块：

- (1) 数据包的捕捉：Netfilter 通过在内核的不同位置设置钩子点，从而在不同点捕捉通过内核的 IP 报文；
- (2) 数据包的过滤：每个钩子点对应一个 hook 链，hook 链中的每个 hook 对应一个功能模块；每个功能模块都会对经过它的 `skb` 依据规则进行处理和过滤；
- (3) 功能模块的注册：每个功能模块都会向 Netfilter 注册自己的 hook，可以同时向多个钩子点注册自己对应的 hook；功能模块主要包括 Iptables(包过滤)和 Conntrack(连接跟踪)两大部分，如 Iptables 就针对不同的钩子点预定义了多个 hook；
- (4) 用户接口的注册：每个功能模块都会向 Netfilter 注册自己的 sock 调用接口，用于用户对规则的管理。

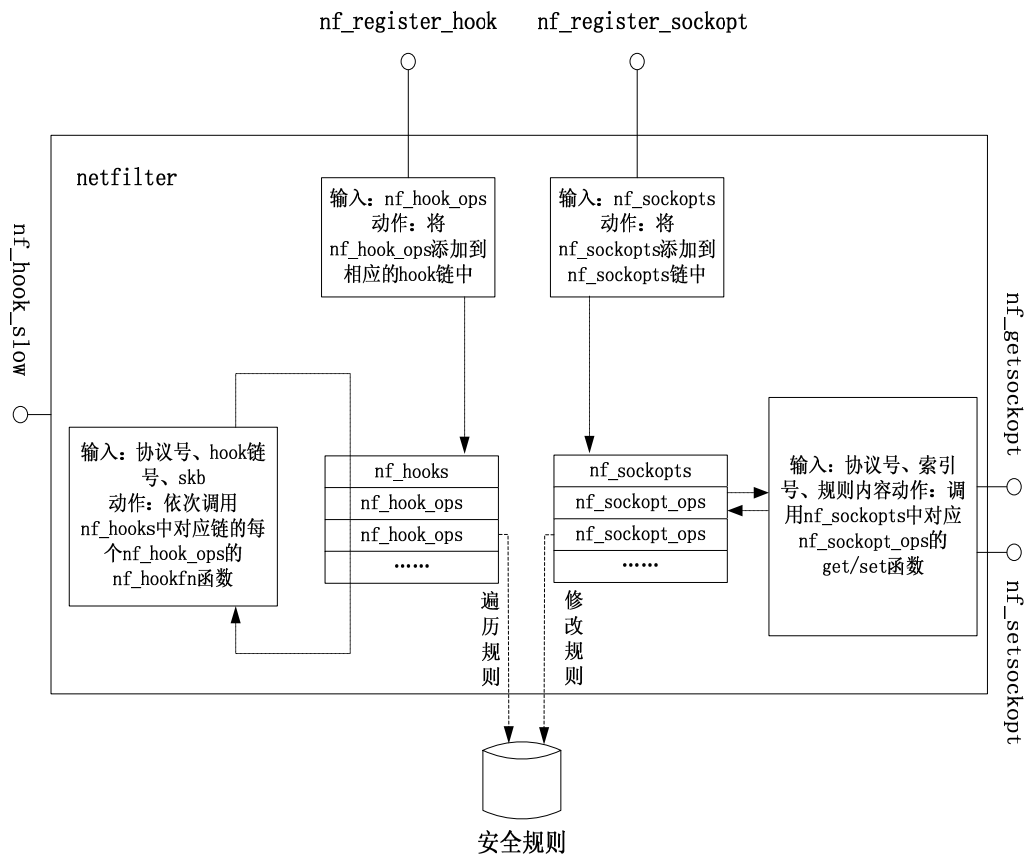


图 2.2 Netfilter 框架

如图 2.2 所示，Linux 内核中 Netfilter 架构主要包含以下几个模块：

- (1) 数据包的捕捉：钩子点，Netfilter 通过在内核的不同位置设置钩子点，从而在不同点捕捉通过内核的 IP 报文；
- (2) 数据包的过滤：每个钩子点对应一个 hook 链，hook 链中的每个 hook 对应一个功能模块；每个功能模块都会对经过它的 skb 依据规则进行处理和过滤；
- (3) 功能模块的注册：每个功能模块都会向 Netfilter 注册自己的 hook，可以同时向多个钩子点注册自己对应的 hook；功能模块主要包括 Iptables(包过滤)和 Conntrack^[16](连接跟踪)两大部分，如 Iptables 就针对不同的钩子点预定义了多个 hook；
- (4) 用户接口的注册：每个功能模块都会向 Netfilter 注册自己的 sock 调用接口，用于用户对规则的管理。

2.2.2 Linux 内核钩子点

如图 2.3 所示，为 Linux 内核设置的 5 个钩子点以及它们之间的对应关系。

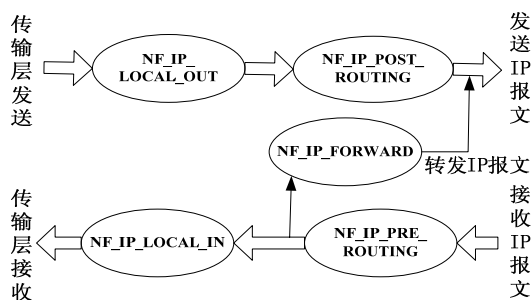


图 2.3 Linux 内核钩子点

2.2.3 hook 的管理

2.2.3.1 hook 数据结构和 hook 链

hook^[17]数据结构和 hook 链结构如图 2.4 所示。每个 hook 都有自己的 hook 处理函数，当钩子点遍历到该 hook 时，就会执行相应的 hook 处理函数。hook 也有自己的优先级，优先级高的 hook 先被遍历，优先级低的 hook 后被遍历。每个钩子点都对应一个 hook 链，每个 hook 链由若干个注册在这个钩子点的 hook 组成。IPv4 和 IPv6 各有 5 个 hook 链。

2.2.3.2 hook 的注册

要将一个 hook 注册到一个 hook 链中，首先定义 nf_hook_ops 结构体，结构体中要指明该 hook 的处理函数、注册的协议簇、hook 链号(Linux 内核中的 5 个钩子点)和该 hook 的优先级，然后调用 nf_register_hook 将该 hook 注册到对应的 hook 链中。注册代码如下：

```
int nf_register_hook(struct nf_hook_ops *reg)
{
    struct list_head *I;
    for (i = nf_hooks[reg->pf][reg->hooknum].next;
        i != &nf_hooks[reg->pf][reg->hooknum];
        i = i->next) {
        if (reg->priority < ((struct nf_hook_ops *)i)->priority)
            break;
    }
    list_add(&reg->list, i->prev);
    return 0;
};
```

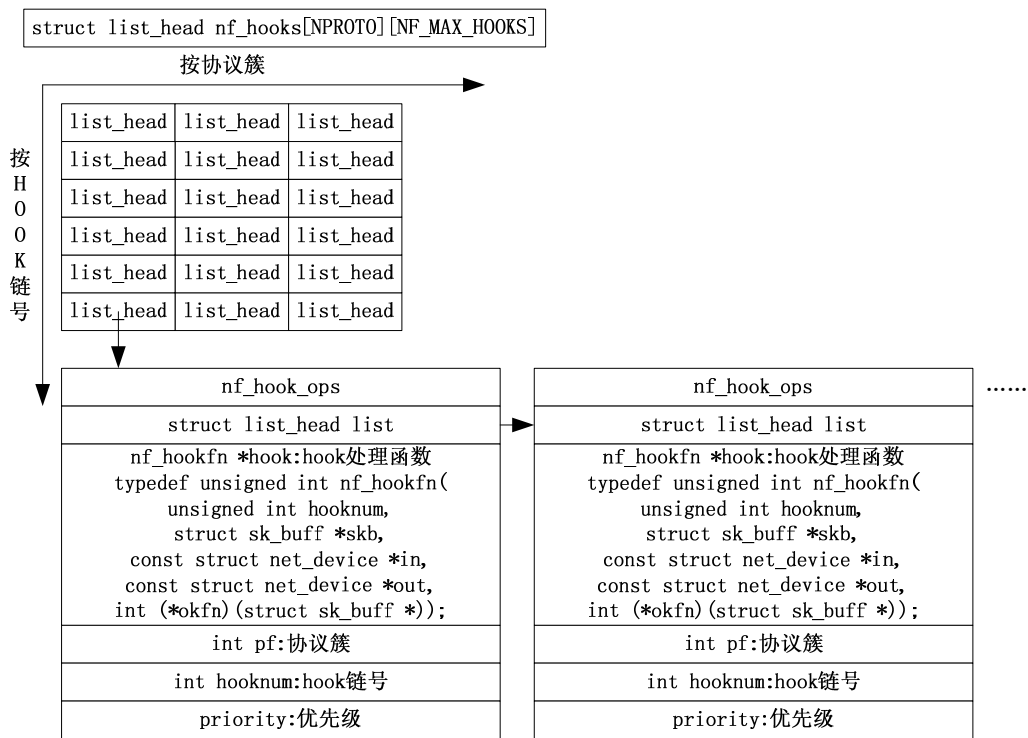


图 2.4 hook 数据结构和 hook 链结构

2.2.3.3 钩子点的处理

我们以 NF_IP_FORWARD 钩子点为例来说明 Linux 内核钩子点的处理流程，如图 2.5 所示：

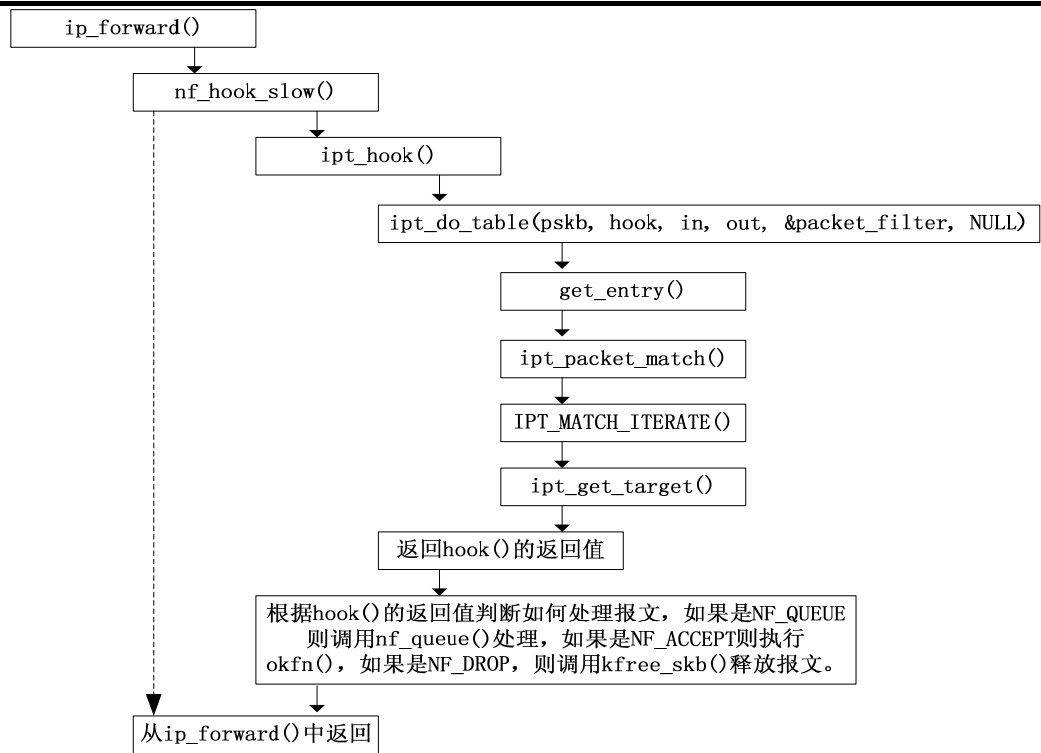


图 2.5 钩子点处理流程图

2.2.3.4 添加一个新的 hook 的过程

如图 2.6 所示为添加一个新 hook 的过程：

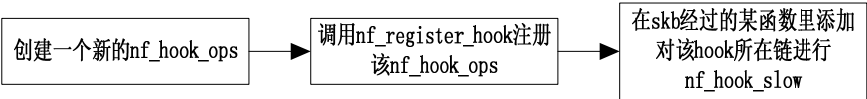


图 2.6 hook 的添加

2.2.3.5 内核中添加一个新 hook 链的过程

如图 2.7 所示，为新增一个 hook 链的的过程。

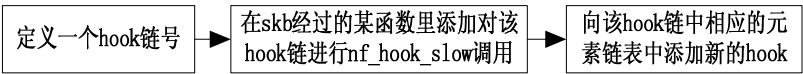


图 2.7 新增 hook 链

说明：采用这种方法添加的链是 Linux 内核中真正意义上的链，与 iptables -N 添加的链不同，后者是虚拟意义的链。

2.2.4 sockopt 的管理

Netfilter 通过数据结构来实现规则的删除、添加、查找。

2.2.4.1 nf_sockopt_ops 数据结构

如图 2.8 所示，Linux 内核中 nf_sockopt_ops 的结构以链表的形式链接起来。

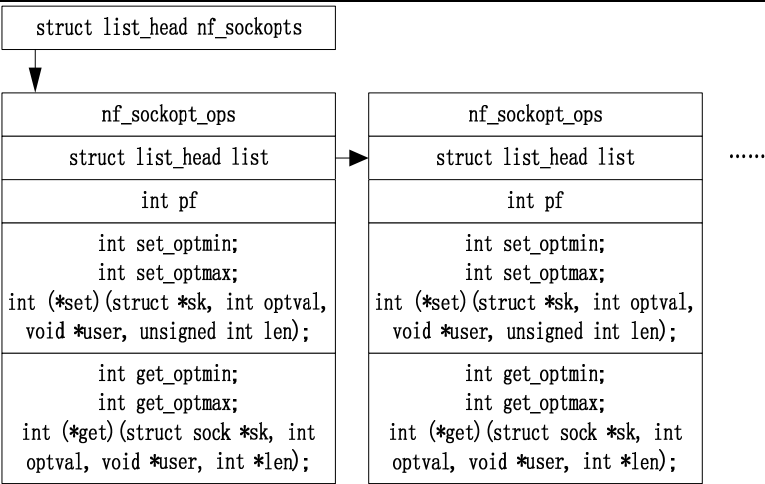


图 2.8 nf_sock_opt 结构

2.2.4.2 nf_sockopt_ops 的注册

每个调用 nf_register_sockopt 注册自己 nf_sockopt_ops 的模块(如 ipt_tables 和 ip_conntrack)在通过 socket 和应用层交互的时候，都会调用到该函数来设置或者读取内容。

2.2.4.3 添加一个新 sockopt 的过程

添加一个新 sockopt 的过程如图 2.9 所示：

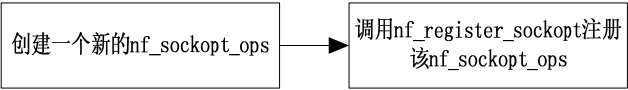


图 2.9 sockopt 添加

2.3 Iptables 机制

2.3.1 Iptables 整体架构分析架构

Iptables 主要实现 Linux 防火墙规则的注册和修改。Iptables 机制主要包含内核模块和用户态模块。内核模块存储了用户态添加的规则以及所有的匹配方法和目标方法。Iptables 内核模块注册在 Linux Netfilter 钩子点上。Iptables 用户态模块通过对用户命令的分析，把它转换成内核能够识别的规则数据结构，最后通过 setsockopt 发送到内核。

如图 2.10 所示，Iptables 包括内核部分和用户态部分。

- (1) 用户通过 Iptables 命令下发字符串数据来代表安全规则数据，Iptables 命令通过用户层匹配和目标方法校验这些字符串数据的有效性并且将这些数据转换成内核匹配和目标方法认识的数据，然后将这些数据保存在用户层匹配和目标中；

- (2) Iptables 命令根据用户层匹配和目标生成一个内核可以认识的新规则；
- (3) Iptables 命令从内核中将当前所有规则读取到用户层构造的 `ipt_replace`，然后将新的规则插入到 `ipt_replace`，接着调用 `socket` 接口下发现有的 `ipt_replace`，从而更新内核的安全规则；
- (4) 内核将受到的安全规则全部保存在内核 `ipt_table` 数据结构中；
- (5) 经过 `hook` 的 `skb` 都将被安全规则过滤，此时也就会调用相应的匹配方法和目标方法，这些方法应该向 Iptables 内核和 Iptables 命令注册，每个方法模块根据规则的数据来对 `skb` 做处理。

2.3.2 Iptables 相关数据结构分析

2.3.2.1 内核中存放安全规则数据的结构

如图 2.11 所示。其中 `ipt_replace` 是用户空间配置程序传递给内核的规则表，这个规则表不能直接使用，必须先根据它里面包含的 `match` 和 `target` 的名称将 `match` 和 `target` 转换成在内核注册的 `match` 和 `target` 的指针，同时检查规则表中是否有循环，如果有循环，要给用户空间的配置程序报告错误。转换之后的规则表存储在 `ipt_table_info` 结构体中。

Iptables 应用程序将命令行输入转换为程序可读的格式，然后根据请求设置了一个 `struct ipt_replace` 结构，其首先将内核中该表所有的规则读出来存放到其中，并在其后附接当前用户设置的这条规则：一个 `struct ipt_entry` 结构体，最后通过系统调用将当前的所有规则下发代替原来的所有规则。

2.3.2.2 内核中处理安全规则处理方法的结构

如图 12 所示，所有的匹配方法都注册到一个 `match` 处理链表中，当处理匹配时都是调用这里注册的处理函数，每个节点实质上由三个函数构成，一个匹配处理函数，一个合法性检查函数，一个析构函数。和 `match` 一样，所有的 `target` 都注册到这个结构类型的全局链表中，每个 `target` 的处理函数都是这里注册的函数。和上面的解释一样，这里也主要包含三个函数指针，`target` 数据结构 `struct ipt_target` 和 `struct ipt_match` 基本相同，不同之处只是用 `target` 函数指针代替 `match` 函数指针，`int (*target)()` 用来执行动作，如果需继续处理则返回 `IPT_CONTINUE(-1)`，否则返回 `NF_ACCEPT`，`NF_DROP` 等值，它的调用者根据它的返回值来判断如何处理该报文。

当某个 `skb` 经过某条 `ipt_entry` 的规则时，会调用到 `ipt_entry` 中的 `ipt_match_entry` 中对于 `ipt_match` 的 `match` 处理函数，该 `ipt_match` 指针就是指向 `ipt_match` 链表中的某个 `match`，从而实现该 `skb` 的匹配处理；`ipt_target` 处理和 `ipt_match` 类似；上述两个链表中的元素是通过 `ipt_register_match` 和

ipt_register_target 来将不同的 match 和 target 加入到其中的。

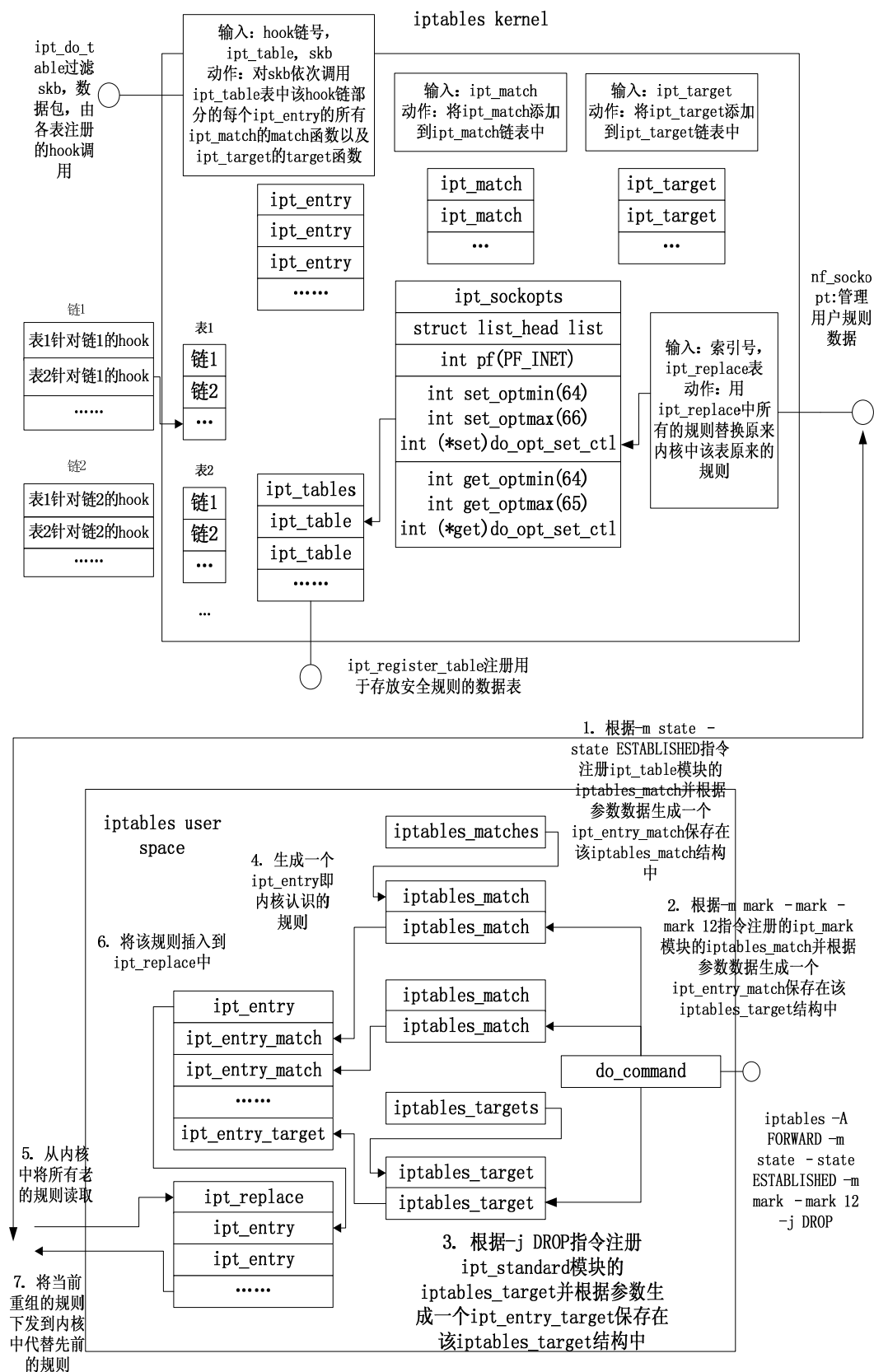


图 2.10 iptables 框架

2.3.2.3 ipt_match 和 ipt_target 的注册函数

ipt_register_match 将匹配处理模块注册到全局链表 ipt_match 中；
ipt_register_target 将目标处理模块注册到全局链表 ipt_target 中。

```
int ipt_register_match(struct ipt_match *match)
{
    list_named_insert(&ipt_match, match);
    return ret;
}
int ipt_register_target(struct ipt_target *target)
{
    list_named_insert(&ipt_target, target);
    return ret;
};
```

2.3.2.4 用户层存放安全规则处理方法的结构

用户层存放安全规则方法的结构如图 2.13 所示，下面以一条命令为例，从 iptables 命令的输入到内核可以认识的规则数据结构的转化过程如下：

- (1) 用户执行 iptables -m mark -mark 12 -j DROP；
- (2) iptables 命令的 do_command 分析 -m mark -mark 12；
- (3) 从 iptables_matches 中查找对应的 iptables_match；
- (4) 没有找到，此时将强行加载 ipt_mark.so 模块，该模块的初始化函数将调用 register_match 来注册 ipt_mark；
- (5) 将 -mark 12 作为参数调用 mark.parse，该函数将参数分析后转化成一个 ipt_entry_match 保存在 ipt_mark 中；
- (6) 针对 -j DROP，do_command 采用类似的方法分析，生成一个 ipt_entry_target 保存在 standard 中；
- (7) iptables 命令将上述 iptables_match 和 iptables_target 中的数据抽取出来组合成一个 ipt_entry。

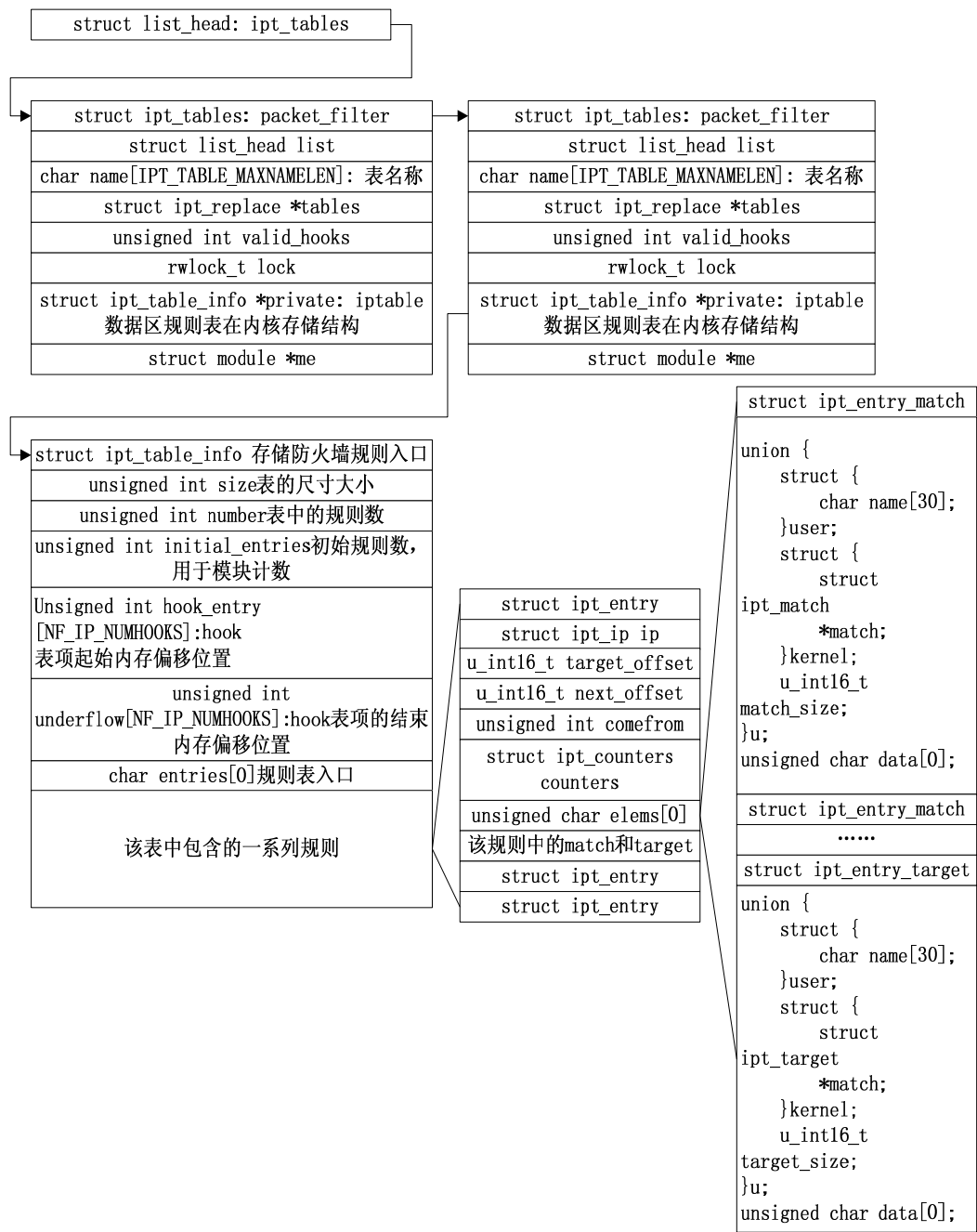


图 2.11 内核中存放安全规则数据结构

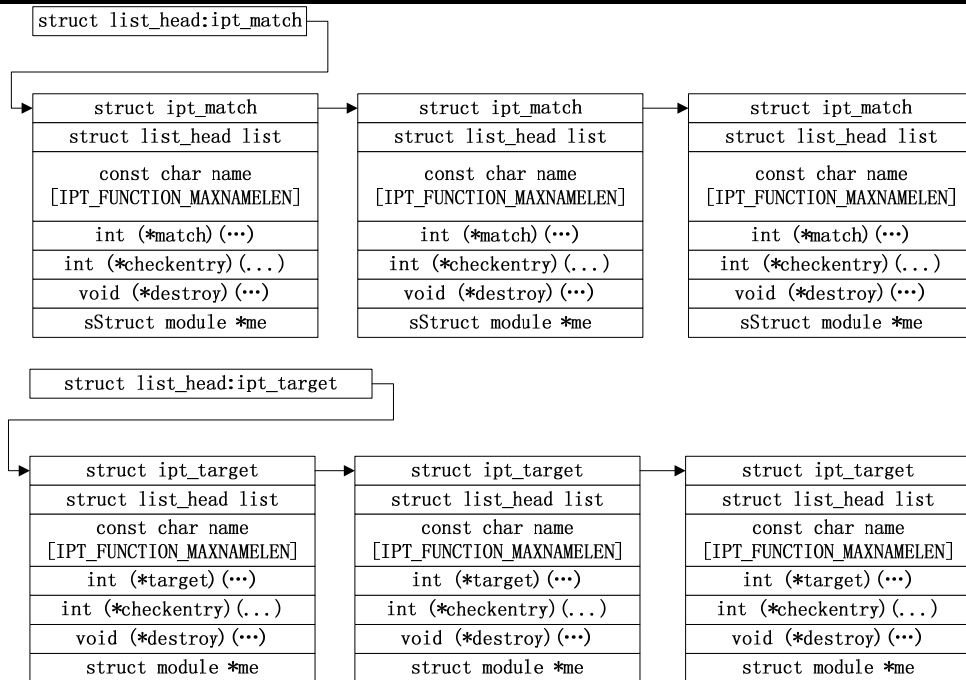


图 2.12 内核中处理安全规则方法的结构

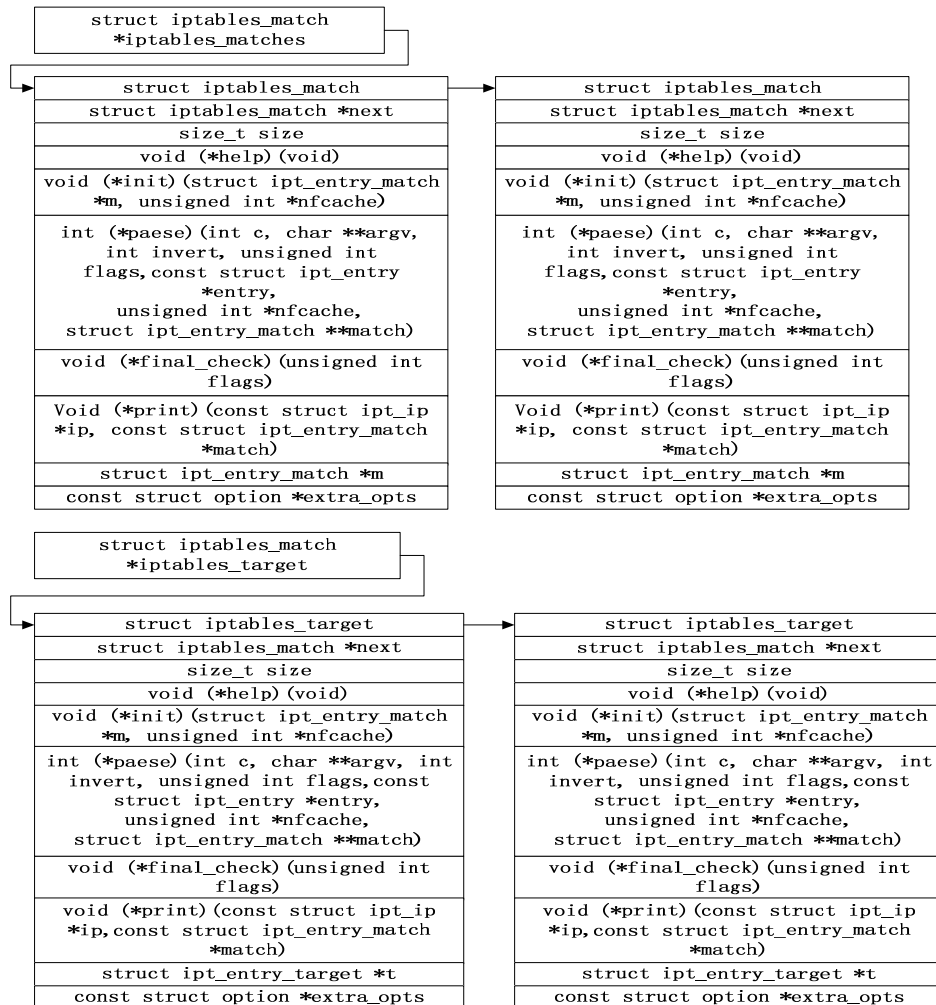


图 2.13 用户层存放安全规则处理方法的结构

2.4 Netfilter/Iptables 内核用户通信机制

Netfilter 还要实现内核与用户之间的通信机制^[18], 使得用户可以在用户态下设置规则。前面提到过 `nf_sockopt_ops` 结构体, Netfilter 在内核中注册 `nf_sockopt_ops` 结构体之后, Linux 内核提供 `copy_from_user()/copy_to_user()` 两个函数是用来提供内核态与用户态数据的拷贝。用户控件通过建立 `socket` 套接字, 使用 `getsockopt/setsockopt` 与内核进行通信。这样基于 Netfilter/Iptables 的防火墙就建立起来了。

2.5 本章总结

本章通过代码分析和调测试相结合的方法, 对 Linux 防火墙架构和处理流程进行了详尽分析。首先讲述了包在 Linux 内核网络协议栈的发送接收流程, 并且重点突出了包在 Netfilter 上的处理, 使得我们对于 Linux 防火墙对实现流程有基本的了解。然后再详细介绍 Netfilter/Iptables 框架, 使得我们对于基于 Netfilter/Iptables 的防火墙实现技术细节有更深入的理解。最后介绍了 Linux 防火墙内核与用户态通信机制。

第三章 IPv6 防火墙高性能规则匹配算法 HiPACv6 研究

前面我们提到过, Iptables 是利用规则的线性匹配来实现的 Linux 防火墙。同样的, 在 IPv6 下, Ip6tables 也是利用线性规则匹配算法来实现 Linux 防火墙。但是当规则数增加时, 利用线性规则匹配算法实现的 Linux 防火墙性能急剧下降, 虽然在 IPv4 时代, 我们有很多针对于 Linux 防火墙的优化算法, 但是考虑到 IPv6 地址的特殊性以及 IPv6 与 IPv4 的很多不同, 这些优化算法都不能直接应用于 IPv6 防火墙。本文通过简单介绍 IPv4 下几种防火墙优化算法, 从中吸取规则的分类思想, 提出了一种适合于 IPv6 下的防火墙优化算法 HiPACv6。

3.1 包分类算法

包分类算法的效率决定着规则匹配效率。包分类是把网络报文划分成不同流的过程。提供包分类的一个系统或者是一个算法叫做包过滤, 或者是包分类器, 或者是流分类器。给定一个规则和一个报文, 当报文头相应域的值包含在规则的匹配范围之内时, 我们说这条规则与这个报文匹配^[19]。

现在, 需要进行包过滤的应用程序越来越多, 主要的动机是把原来因特网最大努力交付服务朝着提供一个安全的、私有的、高性能高可靠性的方向发展。例如, 传统的 IP 转发只是基于目的 IP 地址进行的, 然而这将被策略路由取代, 策略路由除了目的 IP 地址以外同时考虑了源 IP 地址、协议、端口号等。还有其他需要进行包分类的应用例如 VPN、QoS、NAT、防火墙等。包分类是防火墙的基础。不同的应用对包分类系统提出了不同的要求, 评估包分类算法的几个标准是:

- (1) 包分类性能, 许多应用都对包分类性能提出了要求。包分类的性能越高, 网络吞吐率越高;
- (2) 更新性能。包分类算法可以包含静态、增量、动态包分类算法^[20]; 静态包分类算法在更新的时候需要从头开始建立数据结构; 而增量包分类算法则允许动态的插入规则, 而当删除规则的时候需要重建数据结构; 动态包分类算法则允许动态的插入和删除规则;
- (3) 内存消耗: 内存的消耗是包分类算法性能的一个重要指标, 如果包分类算法消耗的内存空间太大, 则会影响到系统的整体性能。可变的内存消耗对于包分类算法显得十分重要;
- (4) 包分类的支持的维数^[21]: 有些算法只支持一维包分类(例如只针对与目的地址), 还有些包分类算法只是支持二维, 也有些算法支持任意维数;

(5) 实现的灵活性：有些算法是专门针对于软件实现而设计的，还有些算法是专门针对于硬件而设计的，有的算法两种都可以。

IPv4 下集中常见的包分类优化算法有 RFC、HiCuts 算法、TSS、TCAM、HiPAC 等。其中 TCAM 是用硬件实现的算法。下面我们在简要介绍前面几种算法之后，将详细分析 HiPAC 算法，并说明 HiPAC 算法的不足之处以及 IPv6 下 HiPAC 算法的瓶颈。

3.2 IPv4 防火墙高速规则匹配算法分析

3.2.1 目前高速规则匹配算法概述

3.2.1.1 RFC 算法

RFC 算法的全称为 Recursive Flow Classification, 是 Pankaj Gupta 和 Nick McKeown 提出的一种基于降维思想的快速包分类算法。该算法就类似于一个映射函数，将不同的数据包映射到不同的种类。它是把一个较大的规则集合映射成较小的等价类集合。包的分类问题被看做成为一个从包头的 S bit 数据到 T bit classID 的映射。映射有两种情况，第一种是单步映射，所谓的单步映射就是预先计算出数据包头的 S 位，并且生成 2^S 种不同情况所对应的 classID 值，这样每个数据包只需要查找一次表就可以得到相应的 classID，但是这样会消耗极大的内存空间。第二种是多步映射，所谓的多步映射就是映射不是通过异步来完成的，而是分为多个阶段来完成的，每个阶段的结果是将一个较大的集合映射成一个较小的集合，RFC 算法正是采用第二种思想。RFC 算法共分为 P 个阶段，每个阶段由一系列并行的查找组成。

RFC 对 IP 报文的查找方法如下：

- (1) 首先数据报文的包头匹配域被分成若干块(俗称为 chunks)，每一块被用来作为并行查找的索引；
- (2) 接下来的几次查找所用的索引值是前面几个阶段的查找结构合并的结果，例如，可以把前面几个查找的结果按二进制位串联起来；
- (3) 在查找的最后，得到一个值，这个值就是该数据报文所对应的 classID。

为了使方法比较直观，下面结合实例对 RFC 进行详细说明：

- (1) 首先对数据包头进行分割，分成 6 块，如表 1 所示，规则表如表 3.1 所示：

表 3.1 匹配域的分块

源 IP 地址		目的 IP 地址		协议号	目的端口号
第 0 块	第 1 块	第 2 块	第 3 块	第 4 块	第 5 块
16bit	16bit	16bit	16bit	8bit	16bit

计算出每块的 S 位, 供 2^S 中不同的情况所分别对应的 eqID 值, 建立一种以 Sbit 的值为索引、eqID 为内容的表, 相同的 eqID 的位 CES(Chunk Equivalence Set)等价块设置。为了方便计算我们引入 CBM(Chunk BitMap)。例如: CBM:100001 表示属于 Rule#1 和 Rule#2。最后得到每块对应的 eqID 和 CBM 表, 用于下一阶段的计算;

- (2) 将第 1 阶段的 0,1,4 块对应的 eqID 与 CBM 表结合, 得到的 Chunk#10, 并得到对应的 eqID 与 CBM 表, 计算 0,1,4 块对应的 eqID 所有组合得到新的 eqID, 利用索引 $\text{index} = \text{eqID}_{\text{chunk0}} * N1 + \text{eqID}_{\text{chunk1}} * N2 + \text{eqID}_{\text{chunk4}}$ 定位, 新的 CBM 有 0,1,4 对应的 CBM 相与操作得到。同理将 2,3,5 块结合, 得到 Chunk#11 及与之相对应的 eqID 与 CBM 表;
- (3) 将第 2 阶段得到的 Chunk#10 与 Chunk#11 结合, 方法同第 2 阶段, 得到最终的 eqID 与 CBM 表, 从 CBM 表中提取出对应的规则号, 从而得到 eqID 表和 RuleID 表, 如表 3.2 所示。

表 3.2 规则的匹配范围和目标

Rule ID	源 IP 地址 (0-15bit)	源 IP 地址 (16-31)	目的 IP 地址 (0-15)	目的 IP 地址 (16-31)	协议号	目的端口号	规则行为
1	0.0/0	0.0/0	0.0/0	0.0/0	*	*	拒绝
2	192.168/16	1.0/8	0.0/0	0.0/0	TCP	*	允许
3	192.168/16	1.73/16	0.0/0	0.0/0	UDP	>1023	允许
4	158.166/16	1.42/16	192.168/16	1.0/8	UDP	20-30	允许
5	0.0/0	0.0/0	10.0/8	0.0/0	ICMP	*	允许
6	10.8/8	0.0/0	158.166/16	1.43/16	TCP	80	允许

3.2.1.2 Hicuts 算法

Hicuts 算法是基于决策树构造的算法, 当每一次有数据包文到达时, 都要查找决策树来找到一个叶子节点, 这个叶子节点中存储了一个小的规则集。最后通过规则的线性匹配来查找匹配的规则。下面我们分析下 Hicuts 算法原理。要分析 Hicuts 算法首先应该明确一下几个概念:

- (1) 规则集中的地址总是以网络地址前缀出现的, 因此, 一条规则中的源地址或目的地址可以表示为一个区间的值;
- (2) 定义: $B(v)$ 域包含一个 K -tuple(K 元组), 每个元素的范围是: $[I1:r1], [I2:r2], \dots, [Ik:rk]$;
- (3) 定义: $C(v)$ 是按照某一维度 d 进行切割。对于 32 为 IP 地址, $C(v)$ 的范围在 $(1 \sim 32)$;

- (4) 定义: $np(C)$ 是按照 $C(v)$ 的切割数, $np(C)=2^{C(v)}$;
- (5) 定义: $R(v)$ 为决策树种根节点的规则集合, $R(u)$ 为 $R(v)$ 子节点的规则几何;
- (6) 定义: $binth$ 为叶子节点中最大规则数, 如果某个节点中的规则数大于 $binth$ 就要继续切割, 知道小于或等于 $binth$ 为止;
- (7) 定义: $spfac$ 为建树时消耗内存的因子。通过 $spfac$ 和 $binth$ 可以控制树的形状, 即树的度和深度。

从根节点开始, 在树每一层节点进行切割, 如果被切割的子域(规则子集) $R(v)$ 中的规则树 大于 $binth$, 则继续切割, 直到 $R(v)$ 中的规则树小于或等于 $binth$ 为止。 $R(v)$ 的规则数小于等于 $binth$ 的节点成为叶子节点 Leaf Node, 大于 $binth$ 的节点成为内部节点。表 3.3 为规则和每个规则的元素的范围。

表 3.3 规则和规则范围

Rule	X range	Y range
R1	0~31	0~255
R2	0~255	128~255
R3	64~71	128~255
R4	67~67	0~127
R5	64~71	0~15
R6	128~191	4~131
R7	192~192	0~255

如图 3.1 所示, 为规则集对 $B(v)$ 域的划分。

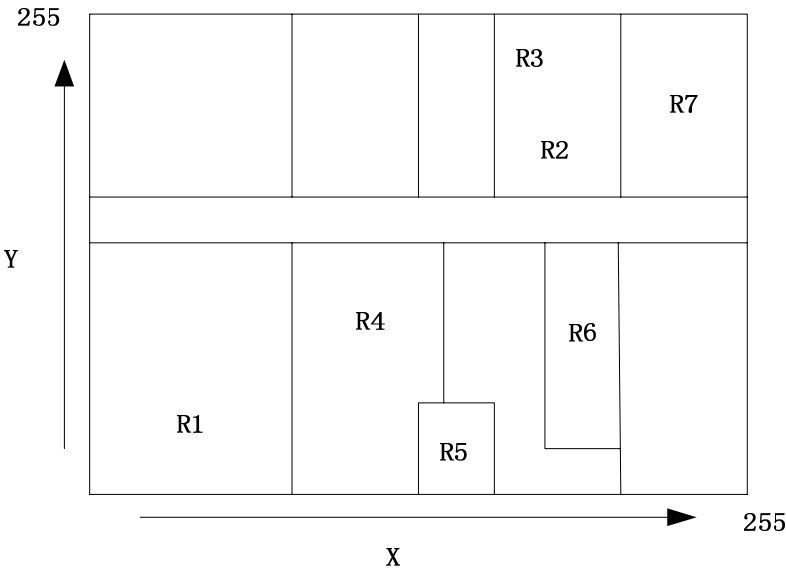


图 3.1 规则集对 $B(v)$ 域的划分

图 3.2 是 HiCuts 算法构造的树的结构示意图。

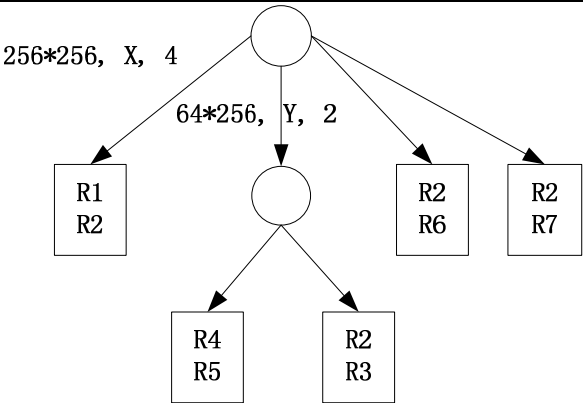


图 3.2 HiCuts 树的构造

我们知道在包分类算法中始终存在时间复杂度和空间复杂度的平衡。在 HiCuts 算法中， $np(C)$ 的数目越大建树的度越大，这样可以缩短每次查找的时间，但相对也增加了内存的消耗。

3.2.1.3 TSS 算法

TSS 算法又叫元组空间搜索算法，该算法适用于前缀规则匹配的分类查找。该算法主要思想是利用实际规则库特性来提高搜索的速度。一般来说，尽管过滤规则数据中含有不同前缀和范围的数量很多，但是不同的前缀长度的数量往往是很小的，因此各字段的前缀长度组合的数目也比较少。基于这种现象长度都相等的规则放在一起，组成一个元组，所有元组的集合成为元组空间。通过这种方式，大规则空间的搜索就转化为较小 Tuple 空间搜索，从而提高了分类效率。在同一个 Tuple 中，规则的各个字段具有固定的前缀长度，所以可以采用 Hash 方法来快速的找到匹配规则。表 3.4 分别是六条规则的元组空间。

表 3.4 规则的元组空间

Rule	F1	F2	元组
R1	00*	00*	(2,2)
R2	0*	01*	(1,2)
R3	1*	0*	(1,1)
R4	00*	0*	(2,1)
R5	0*	1*	(1,1)
R6	*	1*	(0,1)

3.2.1.4 TCAM 算法

TCAM 算法是一种基于内容查找的存储器，它以变量/掩码的形式存储数据项，其中变量掩码都是 W 比特长，这种形式和 IP 地址的表示方法很相似，使得 TCAM 的 IP 路由查找中得到了广泛的应用。对于包分类问题，如果 $W=64$ ，则对于一个二维包分类的规则 (202.197.12.*)、(12.64.*) 在 TCAM 中将以

(202.196.51.0,166.111.0.0,255.255.255.0,255.255.0.0)的形式存储，在进行分类处理时，首先提取数据包相应匹配域的值，然后连接各个值，组成 TCAM 的输入，查询时只需在掩码置 1 的位与变量进行比较匹配。图 3.3 为 TCAM 算法的基本流程。

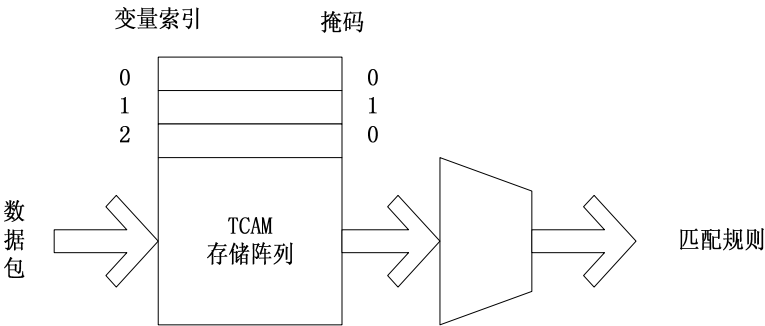


图 3.3 TCAM 算法

首先所有的规则按照优先级降序排列存储于 TCAM 的存储矩阵中，分类查找时，根据数据包的包头信息并行的与所有规则进行匹配，数据 N 位的比特向量，该向量中的各位对应的规则已经预先按优先级进行了降序排列。这样，对于输出的 N 位比特向量只需在通过一个优先级判决的逻辑电路的处理即可得到最佳匹配规则。虽然 TCAM 分类速度很快，但是其价格昂贵，能耗也比较大，因为是由硬件进行并行查找。另外，范围表达的规则在 TCAM 中要展开成为多条前缀规则，会占用很多 TCAM 空间，因此也不适合与大规模的数据库。

3.2.2 HiPAC 算法

3.2.2.1 HiPAC 算法分析

下面我们将详细的介绍 HiPAC 算法。HiPAC 算法是一种高性能 IPv4 防火墙包分类算法，在企业级 IPv4 防火墙有着重要的应用。算法的核心思想是首先把规则进行分类处理，分类是根据规则在各个匹配域的范围把匹配域的整个匹配空间分成若干段，多个匹配域的分段就形成了一个 B+树[22]。要匹配的包通过各个匹配域索引到最后的叶子节点，叶子节点中优先级最高的规则即为匹配的规则。

为了进一步阐明 HiPAC 算法，我们假设有 3 条规则，编号分别为 0,1,2；同时假设有 3 个匹配域(例如协议号、源端口、目的端口)，并且假设每个匹配域的最大匹配范围都是(0,15)；各个规则在各个匹配域的匹配范围如下所示：

- Rule0: (3,8) (0,15) (3,11)
- Rule1: (0,8) (0,8) (3,15)
- Rule2: (3,15) (0,15) (0,11)

假设有一个包，在这三个匹配域的数值分别为(4,6,2)。

由于一个钩子点中所有的规则的集合被称为一个链，一个规则链中都有一个策略规则，表示在所有规则都不匹配的情况下默认操作，所以为了能够让策略规

则能够与所有的包相匹配，策略规则的匹配范围就是各个域的最大匹配范围，即都是(0,15)。

首先根据各个规则在各个匹配域的匹配范围，建立一个 B+树，B+树的构建过程如下所示：

- (1) 首先根据各个规则在协议号域的匹配范围将协议号域范围分成若干个区域(这里是 3 个区域)。如图所示，在第一段区域中，只有规则 1；第二段区域则包含了规则 0、规则 1、规则 2 三条规则；第三段区域只包含了规则 2；
- (2) 根据源端口号域分别对上面分成的三个区域所包含的规则进行分段；
- (3) 如此循环，直到最后一个匹配域；
- (4) 最后在叶子节点中保存着满足各个匹配域范围所构成的索引的规则。如图 3.4 所示，在图中标有椭圆的地方，表示的是满足匹配域 1 范围包含(0,8)、匹配域 2 范围包含(0,8)、匹配域 3 范围包含(3,15)的规则只有规则 1。注意，如果叶子节点中有多条规则，则优先级最高的那条规则即为要找的规则。

B+树构造完成之后，就可以对包进行匹配，包的匹配主要通过查找 B+树来实现。由 B+树的定义可知，B+树中中间节点是导向叶子节点的索引。叶子节点存储的才是有用的信息，本文中叶子节点存储的是指向规则的指针。包(4,6,2)匹配过程是图中标注为叉所构成的路径，具体的匹配过程为：

- (1) 对于协议号来说，包的协议号为 4，落在第二个分段里面；
- (2) 在第二个分段指向的中间节点中对第二个匹配域进行匹配，包的第二个匹配域为 6，落在这个中间节点的第一个分段中；
- (3) 如此反复的执行，直到找到叶子节点为止；
- (4) 最终落在第二个叶子节点的第一个分段中。

如图 3.4 所示，这个分段只包含规则 2，所以找到了与包相匹配的优先级最高的规则为规则 2。最后执行规则 2 的目标。

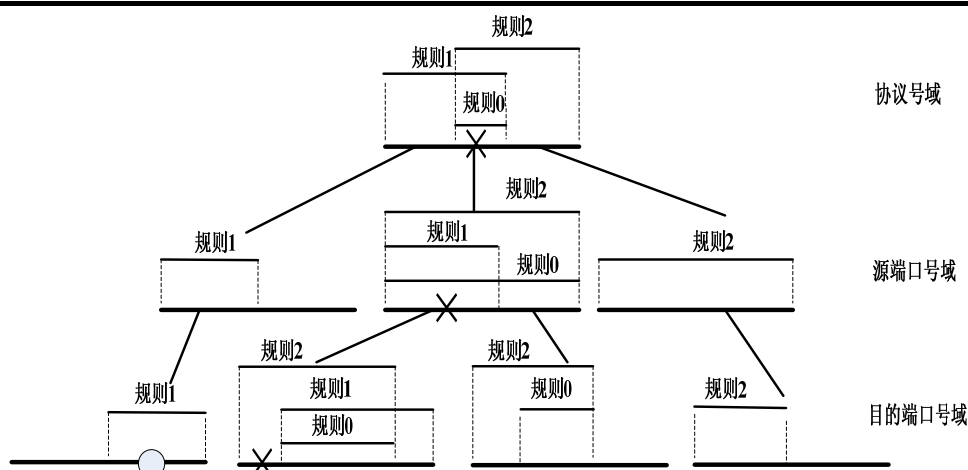


图 3.4 HiPAC 算法以及报文查找过程

HiPAC 算法是一种动态包分类算法，支持规则的动态插入和删除。

3.2.2.2 HiPAC 性能分析

对于 HiPAC 算法而言， n 条规则最坏情况把每个节点分成 $2n$ 段，如果采用二分法查找，每个节点查找次数为 $\log_2(2n)=1+\log_2(n)$ ，所以 HiPAC 算法规则查找时间复杂度为： $T_{\text{time}}=O(\log_2(n))$ 。

HiPAC 算法的空间复杂度是 n 条规则的存储空间和 B+树节点的存储空间，即 $n+(1+2n+(2n)^2+\dots+(2n)^{k-1})$ ，所以 HiPAC 算法的空间复杂度为： $T_{\text{space}}=O(n+(1+2n+(2n)^2+\dots+(2n)^{k-1}))=O(n^{k-1})$ 。我们可以看到 HiPAC 算法空间复杂度随着规则的增加成指数增长。

3.2.3 选择 HiPAC 算法的原因

事实上，包括 RFC、HiCuts 在内的众多 IPv4 高速规则匹配算法都可以作为研究 IPv6 高速规则匹配算法的基础。这里我选择了 HiPAC 算法，主要是因为：

- (1) HiPAC 算法是基于规则范围匹配算法的一个典型代表；
- (2) HiPAC 完全与 Iptables 兼容，也就是说 HiPAC 用户空间命令语法和 Iptables 完全相同，同时 Iptables 规则匹配方法和匹配目标同样适用于 HiPAC；
- (3) HiPAC 算法支持规则的动态更新；
- (4) HiPAC 算法思想对 IPv6 高速规则匹配算法研究有很大的借鉴价值。

3.2.4 IPv6 下 HiPAC 算法的性能瓶颈

根据前面可知，HiPAC 算法主要是各个匹配域的范围，根据规则，把各个匹配域分成若干份，但是这在 IPv6 下是行不通的，因为 IPv6 的地址是 128bit 的，这

个数值很大，如果在按照前面的方法是不可行的。另外，HiPAC 算法的空间复杂度随着规则数的增加呈指数增长，并没有利用好规则的访问空间列表特征。鉴于这种情况，我们提出了一种适合于 IPv6 的高性能包包分类算法，当规则数增加时不会引起空间复杂度的指数增加。

3.3 IPv6 高性能规则匹配算法 HiPACv6

3.3.1 特征分析

3.3.1.1 IPv6 地址特征分析

在 IPv6 核心路由器中，根据 IPv6 的路由表我们可以得到 IPv6 的地址特征^[23]如下：

- (1) 网络号长度分布比较集中，现有的 IPv6 地址网络号的长度大多数都是 24,28,32,48,64，其中，网络号长度为 32 位的占绝大多数。
- (2) 网络号长度超过 64 位的规则很少。
- (3) 规则中 IPv6 地址前缀使用 2001 的占到 70%，而使用 2002 的占到 20%，其他的约占 10%。

3.3.1.2 访问控制列表特征分析

在企业级防火墙或者是核心路由器中，一般可以达到 3000 条规则以上^[24]，虽然如此，一个拥有 5000 条规则的规则集能够匹配一个包的规则数一般不会超过 5 条^[25]。HiPACv6 正是基于这个特点而设计的高性能包匹配算法。

首先我们来分析下 ACL(访问控制列表)特征^[26]：

- (1) 协议号域分析，根据 ACL 规则的特征分析可知，协议域一般包括 TCP、UDP、ICMP 这三种协议号，其他的协议号比较少；
- (2) 端口号分析，通过对 ACL 规则中的源端口与目的端口域特征分析可知：规则中的源端口域绝大多数为“任意”，而目的端口值在大于 1023 的端口一般是用户自定义的端口，在 ACL 规则集中占的比例较少，并且分布不均与。而在 0-1023 之间，由于是操作系统定义的端口，因此端口值的分布也比较均匀。

3.3.2 IPv6 高性能规则匹配算法基本思想

根据以上 IPv6 地址特征和访问控制列表特征，我们可以概括的说明算法的基本思想：

- (1) 由于协议号一般只包括 TCP、UDP、ICMP 协议，所以我们首先利用协议号对规则进行过滤，可以把规则分成 4 类，TCP 规则集、UDP 规则集、

ICMP 规则集、其他协议规则集；

- (2) 把根据协议号过滤的规则集分别在用目的端口号进行过滤，由于目的端口号在 1023 以上的比较少，并且在 0-1023 之间是呈均与分布的，所以我们将目的端口号分成 9 类，其中在 0-1023 之间分成 8 类，最后一类为 1024-65535 为一类。这样经过协议域和目的端口号的分类就把一个规则集分成 20 个子规则集。其中 TCP、UDP 协议各有 9 个规则子集，由于 ICMP 和其他协议没有端口号，所以 ICMP 和其它协议各占一个规则集；
- (3) 利用位选取法把这 20 个规则子集各自分成一个个不大于 5 条规则的规则子集；
- (4) 包经过上述分类走向后，落到一个或者多个不大于 5 条规则的规则集中，再将包线性匹配规则。这样折中了范围匹配算法和线性匹配算法的优点。使系统能够在内存消耗不是很高的情况下达到较高的吞吐率。

如图 3.5 所示，为算法基本结构。

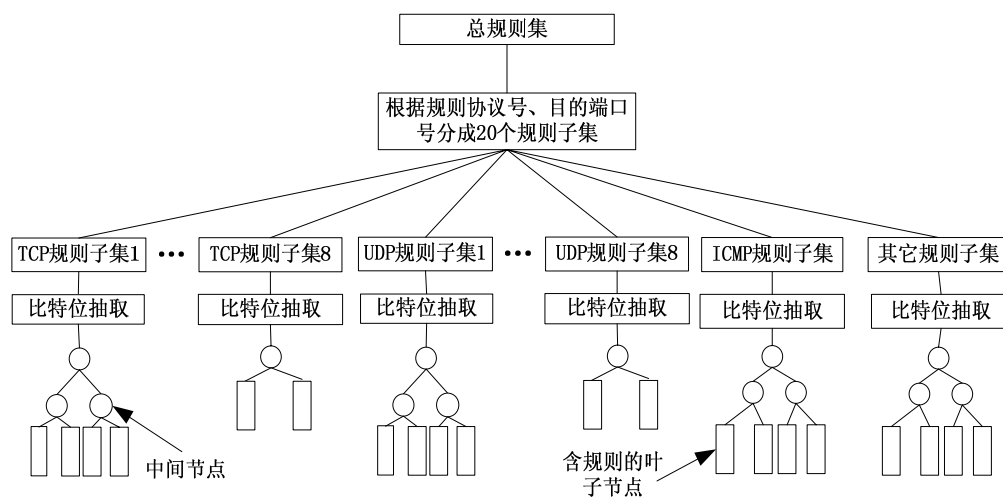


图 3.5 HiPACv6 算法结构图

3.3.3 位选取法

一个大规则集经过协议号和目的端口号的分类将整个规则集分成 20 个规则子集，每个规则子集再根据源 IP 地址目的 IP 地址的位选取法分成一个个不大于 5 条规则的小的规则集合。

比特位选取：由前面可知，以 2001 开始的 IPv6 地址占现阶段整个 IPv6 地址的 70%，所以 IPv6 地址前面 16 个 bit 位对规则区分度不大，并且网络号长度为 32 为占绝大多数。因此我们把比特位的选取限定在源 IPv6 地址的 16~31 位和目的 IPv6 地址的 16~31 位。针对这 32 个 bit 位，假设对 m 条规则进行位选取， n_{i1} 表示的是在第 $i(0 < i < 32)$ 个比特位上为 1 的规则个数， n_{i0} 表示的是在第 i 个比特位上为 0

的规则个数。 $|n_{i1} - n_{i0}|$ 表示两者之间差值的绝对值。比特位选取的位置就是使 $|n_{i1} - n_{i0}|$ 最小对应的 i 。

被分成的 20 个子集构造树的过程如下:

- (1) 首先将规则的源 IP 地址和目的 IP 地址拼在一块, 构成一个 256 位数(注意, 每个 位有三种取值, 0、1、*);
- (2) 查找树, 判断该节点是否中叶子节点, 如果是, 并且叶子节点中规则数小于预定义的大小 N , 则直接把规则放入叶子节点中, 叶子节点规则数增加 1, 规则插入结束;
- (3) 如果是叶子节点, 但是叶子节点中的规则数等于预定义的值 N , 则需要选取特定的比特位把叶子节点中的规则和将要插入的规则分在两个新建的叶子节点中, 原来的叶子节点变成中间节点, 成为索引叶子节点的一部分。选取比特位位置后, 把这些规则按照规则在这个比特位的值来分, 如果规则在这个比特位是 1, 则规则被分到左叶子节点, 如果规则在这个比特位为 0, 则被分到右叶子节点, 如果规则在这个比特位上是 *, 则规则既要被分配到左叶子节点, 又要被分配到右叶子节点;
- (4) 如果节点不是叶子节点, 是中间节点, 则根据中间节点存储的比特位位置, 如果规则在这个比特位是 1, 则继续匹配左子节点; 如果这个比特位是 0, 则继续匹配右子节点, 如果规则在这个为上是*, 则规则分两条路径, 左子节点和右子节点都要匹配。转到流程(2);
- (5) 对于每个子规则集中所有的规则都进行上面的插入操作, 构建 20 个二叉树。

3.3.4 包的匹配流程

包的匹配流程:

- (1) 首先根据包的协议号和目的端口号确定包落在那个规则子集中, 查找这个规则集对应的二叉树;
- (2) 如果节点为叶子节点, 则直接线性匹配叶子节点所有规则, 匹配终止;
- (3) 否则如果节点是中间节点, 根据中间节点存储的 bit 位位置(这里把源 IP 地址的 16~31 定义为 0~15, 把目的 IP 地址的 16~31 位定义为 16~31), 判断包的源或者目的 IP 地址在这个 bit 位上是 1 还是 0, 如果是 1, 则找到孩子节点, 如果是 0, 则找到右子节点, 转到流程(2)。

3.3.5 HiPACv6 算法实例分析

为了进一步阐明算法, 我们假设有 10 条规则, 这 10 条规则各个匹配域(源 IP

地址、目的 IP 地址、源端口、目的端口、协议号)如表 3.5 所示:

表 3.5 IPv6 规则匹配范围

匹配域 规则号	协议号	源 IP 地址	目的 IP 地址	源端口号	目的端口号
R0	TCP	2001:1000:::45/32	2001:3564::43/32	1024-65535	22
R1	TCP	2001:1034::24/32	2001:6935::25/32	任意	23
R2	TCP	2001:1600:2356:4956::32/64	2002:6364::25/32	任意	80
R3	TCP	2002:4895::12/32	2001:3698::79/32	任意	25-750
R4	UDP	2001:3578::37/32	2001:4679:9689:3264::35/64	任意	1024-65535
R5	UDP	2001:3265:6842:3264::45/64	2001:6000::78/32	任意	任意
R6	UDP	2001:6542::21/32	2001:1000::67/32	1024-65535	20-600
R7	UDP	2001:4689::16/48	2001:8000::79/32	任意	560-812
R8	ICMP	2001:3465::65/32	2001:6400::29/32		
R9	OSPF	2001:9865::56/32	2001:9000::89/32		

- (1) 首先根据协议域和目的端口域把规则放入相应的规则子集中。例如 R0 的协议号是 TCP，目的端口号是 22，则 R0 属于规则子集 0，各个规则在规则子集中的分布如表 3.6 所示；
- (2) 经过前面的划分，10 条规则就被分到 20 个不同的规则子集。在对源 IPv6 地址和目的 IPv6 地址的串接进行位选取，构造二叉树。这里为了说明位选取法，我们假设叶子节点中的规则数不大于 3 条。下面我们以前规则子集 0 的二叉树创建为例来说明二叉树构造的过程。规则子集 0 中各个规则源 IP

地址和目的 IP 地址的串接之后为表 3.7 所示;

表 3.6 规则分配到的规则集

规则子集 0	R0、R1、R2、R3	规则子集 10	R5、R6
规则子集 1	R3	规则子集 11	R5、R6
规则子集 2	R3	规则子集 12	R5、R6
规则子集 3	R3	规则子集 13	R5、R6、R7
规则子集 4	R3	规则子集 14	R5、R7
规则子集 5	R3	规则子集 15	R5、R7
规则子集 6		规则子集 16	R5
规则子集 7		规则子集 17	R4、R5
规则子集 8		规则子集 18	R8
规则子集 9	R5、R6	规则子集 19	R9

表 3.7 规则源 IP 地址与目的 IP 地址串接

R0	2001:1000:::45/32:2001:3564::43/32
R1	2001:1034::24/32:2001:6935::25/32
R2	2001:1600:2356:4956::32/64:2002:6364::25/32
R3	2002:4895::12/32:2001:3698::79/32

- (3) 提取各个规则中的位选取比特位(源 IPv6 地址的 16~31 位和目的 IPv6 地址的 16~31 位), 用二进制进行表示, 如表 3.8 所示;

表 3.8 比特位提取

R0	0001000000000000000011010101100100
R1	00010000001101000110100100110101
R2	000101100000000000110001101100100
R3	01001000100101010011011010011000

- (4) 为这个规则子集创建一个二叉树, 首先为二叉树分配一个叶子节点的空间, 分别将前三条规则直接添加到叶子节点(叶子节点的规则数不超过三条);
- (5) 由于前面已经向叶子节点中添加了 3 条规则, 所以当插入第四条规则时, 规则需要重新分到两个叶子节点中。利用位选取法进行位选取, 首先计算各个位中 1 的个数和 0 的个数差的绝对值 $|n_{i1}-n_{i0}|$ 。得到结果如表 3.9 所示(其中 $n(i)$ 表示第 i 个比特位 1 的个数和 0 的个数差的绝对值);
- (6) 由比特位选取法可知, 我们需要选取的是第一个 $|n_{i1}-n_{i0}|$ 最小的比特位。根据表 3.9 的数据, 可以得到选取的比特为 $n(11)$, 也就是源 IPv6 地址中的第 27 位。比特位的选取完毕;

表 3.9 比特位 0 和 1 个数差的绝对值

n(0)	4	n(8)	2	n(16)	4	n(24)	2
n(1)	2	n(9)	4	n(17)	0	n(25)	0
n(2)	4	n(10)	2	n(18)	0	n(26)	2
n(3)	2	n(11)	0	n(19)	0	n(27)	0
n(4)	2	n(12)	4	n(20)	2	n(28)	2
n(5)	2	n(13)	0	n(21)	2	n(29)	2
n(6)	2	n(14)	4	n(22)	0	n(30)	4
n(7)	4	n(15)	2	n(23)	2	n(31)	2

- (7) 由比特位选取法可知，我们需要选取的是第一个 $|n_{i1} - n_{i0}|$ 最小的比特位。根据表 3.9 的数据，可以得到选取的比特为 $n(11)$ ，也就是源 IPv6 地址中的第 27 位。比特位的选取完毕；
- (8) 分配一个中间节点，把原来的叶子节点用中间节点代替，并且利用中间节点记录下该次选取的比特位位置(源 IPv6 地址第 27 位)。根据选取的比特位的值，重新把原来的规则和即将插入的规则分配到两个叶子节点中。假设新分配的叶子节点分别是 Y1 和 Y2，源 IPv6 地址第 27 位为 0 时被分配到 Y1，为 1 时被分配到 Y2。分配的结果如表 3.10 所示；

表 3.10 规则的重新分配

规则	源 IP 第 27 位	叶子节点
R0	0	Y1
R1	1	Y2
R2	0	Y1
R3	1	Y2

规则的重新分配如图 3.6 所示。

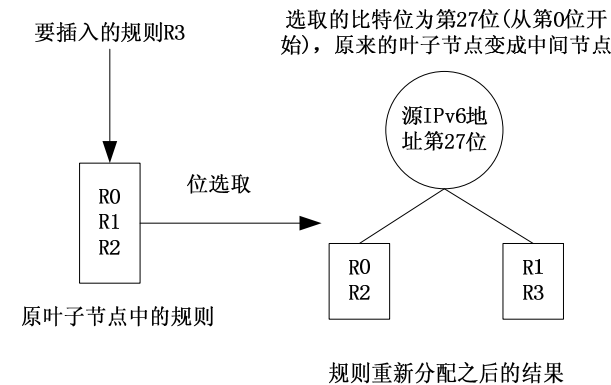


图 3.6 规则的插入

其他规则子集二叉树构建方法与前面一样。这里就不再重复。现在我们假设有这样的一个报文，它的各个匹配域的值如表 3.11 所示：

表 3.11 包各个匹配域的值

匹配域 报文	协议 号	源 IP 地址	目的 IP 地址	源端口号	目的端 口号
packet0	TCP	2001:1034::24/32	2001:6935::25/32	25646	23

- (1) 首先根据包的协议号 TCP 和目的端口号 23 计算出报文属于规则子集 0;
- (2) 由包的源 IP 地址可知, 源 IPv6 地址第 27 位为 1, 所以最终包匹配到 Y2 叶子节点;
- (3) 利用线性匹配的方法将叶子节点中的规则与包进行匹配, 优先级最高的规则即为匹配的规则。所以最终与 packet0 相匹配的规则为 R1。

3.4 本章总结

本章首先介绍了包分类算法的基本概念, 然后简单地介绍了几种经典的 IPv4 下防火墙优化算法, 并重点分析了 HiPAC 算法, 突出了 HiPAC 算法在 IPv6 环境下的瓶颈。针对 IPv6 地址特征和 IPv4 与 IPv6 的区别, 提出了一种适合于 IPv6 环境的高速规则匹配算法。本算法有效借鉴了 HiPAC 和 Iptables 的优点, 结合了 HiPAC 的规则分类以及分段思想和 Iptables 的线性匹配优点(规则数较少情况下效率比较高), 为实现基于 IPv6 的高性能防火墙奠定了算法基础。

第四章 系统实现与关键技术分析

本章主要介绍基于 HiPACv6 算法的 IPv6 防火墙原型系统实现技术。首先对 Linux 内核编程基本知识和技巧进行简要介绍,然后设计了系统总体结构,着重阐述了用户模块和内核模块的系统实现技术,并对关键技术及难点进行了分析。原型系统的实现是基于 Linux-2.6.18 内核完成的。

4.1 Linux 内核编程简介

4.1.1 内核编程与用户态编程比较

- (1) 用户态编程时可以用到很多已经写好的 C 库,但是内核编程不能使用 C 库。在 Linux 内核编程中除了 C 语言本身关键字外,所有用到的函数如果不是自己写的,就要有相关的头文件包含或者是外部引用;
- (2) 用户态访问内存时,有内存保护机制,当访问到一个非法内存时,会引发段错误,但是内核编程则没内存保护机制,当访问有误时,直接会导致系统崩溃;
- (3) 用户态编程可以用浮点数,但是内核编程很少甚至不用浮点数^[27];
- (4) 由于内核支持异步中断^[28]、抢占、SMP^[29],因此必须时刻注意同步与并发。

4.1.2 进程上下文与中断上下文区别

进程最重要的部分之一就是执行程序代码,代码从可执行文件中读取在程序的地址空间中执行。一般的程序在用户空间中执行。当一个程序执行一个系统调用或者触发一个异常时,程序就进入了内核空间。这时候,内核就被认为在进程上下文^[30]。当退出内核时,进程恢复到用户空间的执行。所有对内核的访问都要通过 Linux 提供的系统调用和异常处理接口^[31]。

中断是操作系统管理硬件的重要的方法。不同的设备可以与不同的中断相关^[32]。处理器响应中断的方式就是执行该中断注册的处理函数。当处理器在执行一个中断处理函数时,内核被叫做在中断上下文。在中断上下文的内核,与任何一个进程无关。中断上下文的执行时间非常重要,引文中断处理器的执行是中断了其他代码的执行,所以执行在中断上下文的代码必须又快又简单。

中断包含硬中断和软中断,硬中断主要完成一些紧迫的处理,而有些东西可以尽量推迟一点处理,则可以用软中断来实现。比如说,网卡接收报文,当网卡接收到报文时,应该尽快的将接收到的报文拷贝到内存中,以免新到的报文由于

网卡自带存储空间不足而被丢弃。而当放到内存之后的包处理则可以推迟一点，这就可以用软中断来实现。Linux 内核定义了 6 种软中断，定时器软中断、网卡接收包软中断、网卡发送包软中断、tasklet 软中断、BLOCK_SOFTIRQ 软中断、HI_SOFTIRQ 软中断。

运行在进程上下文的内核是可以睡眠的，可以通过其它进程唤醒，或者是子定义进程的睡眠时间，时间终止后自动唤醒。运行在中断上下文(软中断上下文或者是硬中断上下文)的内核是不能睡眠的。所以运行在中断上下文的代码不能调用会阻塞的函数或者睡眠。

4.1.3 内核编程同步机制

在一个共享的内存应用程序中，开发者必须警惕资源的并发访问。内核编程也不例外。共享的资源必须防止并发访问，因为可能有多线程访问。内核的并发访问可能由以下几种情况导致：

- (1) 中断——中断在任意的时刻都可能中断其他代码的执行；
- (2) 软中断和任务——内核在任意时刻都可能生成一个软中断或者任务，中断当前代码的执行；
- (3) 内核的优先级——由于 Linux 内核进程有优先级，所以一个任务可能抢占另外一个任务的执行；
- (4) 对称多处理器——两个或者多个处理器同时在执行内核代码。

内核开发者必须懂得这些情况并且阻止并发性访问的发生。Linux 内核常用的同步机制如下：

- (1) Per-CPU variables：把数据结构给每个 CPU 拷贝一份；
- (2) 原子操作：原子的读-修改-写；
- (3) 自旋锁：用忙等待来锁变量；
- (4) 信号量：用阻塞等待来锁变量；
- (5) seqlocks：基于访问计数器的锁；
- (6) 关闭本地中断：在单个 CPU 上禁止执行中断处理；
- (7) 关闭本地软中断：禁止软中断函数在单个 CPU 上执行；
- (8) 读-拷贝-更新：通过指针访问共享的数据。

4.1.4 内核与用户态通信机制

实现 Linux 内核用户通信的方法主要包含以下几个：

- (1) procfs 文件系统，proc 文件系统是一个虚拟文件系统，通常挂载在 /proc 目录下。它允许内核通过文件的方式把内核信息导出到用户空间。然而这个

文件并不是真正的存储在磁盘上。通过 `cat` 命令可以读文件，通过 `>shell` 重定向符可以对文件进行写操作；

- (2) `sysctl` 方式，这种借口允许用户空间读和修改内核的变量值。在用户空间下，可以通过两种方式访问这些变量。一种是 `sysctl` 系统调用，另外一种 `procfs` 文件系统；
- (3) `sysfs` 方式，`sysfs` 以一种有组织的方式导出很多信息；
- (4) `ioctl` 系统调用；
- (5) `netlink socket`, `netlink socket` 是最新的也是最常用的用户空间与内核通信的方法。

4.1.5 内核内存管理函数

Linux 内核内存分配函数使用 `kmalloc` 函数进行内存分配。在不同情形下或者不同的上下文，内存分配函数的标志位也使用不一样。下面给出各种情况下内存分配函数标志位的使用：

- (1) 进程上下文、可以睡眠，用 `GFP_KERNEL`；
- (2) 进程上下文、不可以睡眠，用 `GFP_ATOMIC`；
- (3) 中断上下文，用 `GFP_ATOMIC`；
- (4) 软中断上下文，用 `GFP_ATOMIC`；
- (5) `Tasklet`，用 `GFP_ATOMIC`；
- (6) 需要用于 DMA 的内存，可以睡眠，用 `GFP_DMA|GFP_KERNEL`；
- (7) 需要用于 DMA 的内存，不可以睡眠，用 `GFP_DMA|GFP_ATOMIC`。

另外一个用于内存分配的函数是 `vmalloc` 函数，`vmalloc` 函数与 `kmalloc` 函数最大的不同在于，`vmalloc` 函数分配的虚拟地址是连续的，而物理地址则无需连续。而 `kmalloc` 函数分配的内存存在物理地址上是连续的，在虚拟地址上自然是连续的。大多数情况下，只有硬件设备需要得到物理地址连续的内存，硬件设备用到的任何内存区都必须是物理上连续的块，而不是虚拟地址连续的块。由于 `vmalloc` 只要在虚拟地址上是连续的，而真正的物理内存不一定是连续的，所以当需要分配很大的内存块时，可以用 `vmalloc` 以防物理内存没有这么大的连续内存块而导致内存分配失败。

4.1.6 内核开发调试

调试是软件开发过程中一个必不可少的环节，在 Linux 内核开发的过程中也不可避免地会面对如何调试内核的问题。Linux 内核开发调试主要有两种方法，一种是采用打印 `printk` 调试，另外一种是采用类似 `kgdb`^[33] 调试工具进行调试。

下面我们简单的介绍下利用 kgdb 工具进行调试, 然后详细的介绍下 printk 打印调试。

kgdb 提供了一种使用 gdb 调试 Linux 内核的机制。使用 KGDB 可以象调试普通的应用程序那样, 在内核中进行设置断点、检查变量值、单步跟踪程序运行等操作。使用 kgdb 调试时需要两台机器, 一台作为开发机(Development Machine), 另一台作为目标机(Target Machine), 两台机器之间通过串口或者以太网口相连。串口连接线是一根 RS-232 接口的电缆, 在其内部两端的第 2 脚(TXD)与第 3 脚(RXD)交叉相连, 第 7 脚(接地脚)直接相连。调试过程中, 被调试的内核运行在目标机上, gdb 调试器运行在开发机上。安装 kgdb 调试环境需要为 Linux 内核应用 kgdb 补丁, 补丁实现的 gdb 远程调试所需要的功能包括命令处理、陷阱处理及串口通讯 3 个主要的部分。kgdb 补丁的主要作用是在 Linux 内核中添加了一个调试 stub。调试 stub 是 Linux 内核中的一小段代码, 提供了运行 gdb 的开发机和所调试内核之间的一个媒介。gdb 和调试 stub 之间通过 gdb 串行协议进行通讯。gdb 串行协议是一种基于消息的 ASCII 码协议, 包含了各种调试命令。当设置断点时, kgdb 负责在设置断点的指令前增加一条 trap 指令, 当执行到断点时控制权就转移到调试 stub 中去。此时, 调试 stub 的任务就是使用远程串行通信协议将当前环境传送给 gdb, 然后从 gdb 处接受命令。gdb 命令告诉 stub 下一步该做什么, 当 stub 收到继续执行的命令时, 将恢复程序的运行环境, 把对 CPU 的控制权重新交还给内核。

除了 kgdb 调试之外, Linux 内核最常用调试方法就是 printk 打印调试。printk 的日志级别定义如下:

```
#define KERN_EMERG <0> /* 紧急事件消息, 系统崩溃之前提示, 表示系统不可用 */
#define KERN_ALERT <1> /* 报告消息, 表示必须立即采取措施 */
#define KERN_CRIT <2> /* 临界条件, 通常涉及严重的硬件或软件操作失败 */
#define KERN_ERR <3> /* 错误条件, 驱动程序常用 KERN_ERR 来报告硬件的错误 */
#define KERN_WARNING <4> /* 警告条件, 对可能出现问题的情况进行警告 */
#define KERN_NOTICE <5> /* 正常但又重要的条件, 用于提醒。常用于与安全相关的消息 */
#define KERN_INFO <6> /* 提示信息, 如驱动程序启动时, 打印硬件信息 */
#define KERN_DEBUG <7> /* 调试级别的消息 */
```

日志级别的范围是 0~7, 没有指定日志级别的 printk 语句默认采用的级别是 DEFAULT_MESSAGE_LOGLEVEL。内核可把消息打印到当前控制台上, 可以

指定控制台为字符模式的终端或打印机等。默认情况下，“控制台”就是当前的虚拟终端。为了更好地控制不同级别的信息显示在控制台上，内核设置了控制台的日志级别 `console_loglevel`。 `printk` 日志级别的作用是打印一定级别的消息，与之类似，控制台只显示一定级别的消息。当日志级别小于 `console_loglevel` 时，消息才能显示出来。如果系统运行了 `klogd` 和 `syslogd`，则无论 `console_loglevel` 为何值，内核消息都将追加到 `/var/log/messages` 中。如果 `klogd` 没有运行，消息不会传递到用户空间，只能查看 `/proc/kmsg`。

4.2 HiPACv6 算法的 Linux 内核实现

4.2.1 系统总体结构

如图 4.1 所示，系统的实现主要包含用户态部分和内核态部分。用户态部分主要是分析用户输入的命令，把命令转换为规则数据结构，然后通过 `netlink` 发送到内核。内核部分由多个模块组成，他们分别是：`netlink` 接收模块、二叉树操作(插入、删除规则)模块、包匹配处理模块等组成。

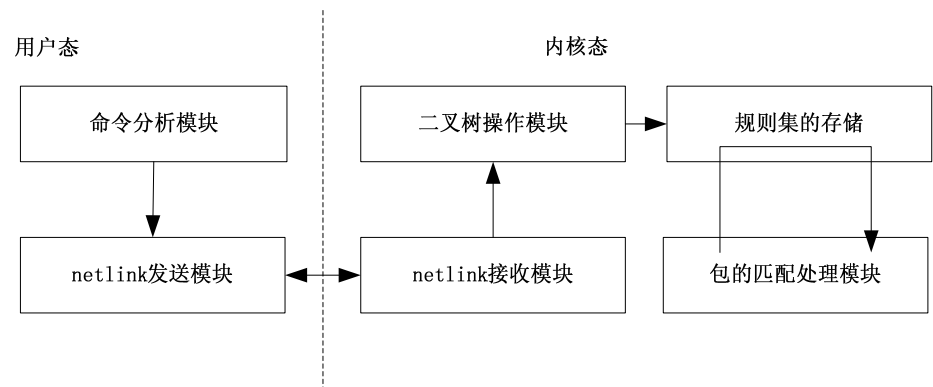


图 4.1 系统组成以及模块之间的关系

4.2.2 用户态部分实现

用户态部分实现主要涉及到命令的分析，通过命令的分析把相应的命令转换成规则结构。以下是定义的规则数据结构：

```
typedef struct {
    unsigned char left;      /* 端口范围左边界 */
    unsigned char right;     /* 端口范围右边界 */
}port;
```

```
typedef struct {  
    unsigned int left[4];    /* IPv6 地址范围左边界 */  
    unsigned int right[4];   /* IPv6 地址范围右边界 */  
}ipv6_address;
```

```
typedef struct {  
    unsigned char protocol; /* 规则的协议域 */  
    port src_port; /* 规则的源端口匹配范围 */  
    port dst_port; /* 规则的端口匹配范围 */  
    ipv6_address src_address; /* 规则的源地址匹配范围 */  
    ipv6_address dst_address; /* 规则的地址匹配范围 */  
    unsigned char src_addr_prefix_len; /* 规则源 IP 地址前缀长度 */  
    unsigned char dst_addr_prefix_len; /* 规则目的 IP 地址前缀长度 */  
    unsigned char action; /* 规则的行为，表示当某个包匹配这个规则后执行的目标  
                           (允许通过、丢弃等) */  
}rule;
```

```
typedef struct {  
    int cmd; /* 当 cmd == 1 时表示增加规则，当 cmd == 0 时表示删除规则，  
              当 cmd == 2 是表示删除所有的规则 */  
    rule rule_data; /* 当命令为插入规则时，解析的规则数据结构体 */  
}command_msg;
```

- (1) 用户按照要求的格式输入命令；
- (2) 程序对输入的命令进行分析得到规则数据结构；
- (3) 创建 netlink 套接字消息，通过 netlink 消息吧规则数据发送到内核。

4.2.3 内核部分实现

4.2.3.1 内核相关数据结构分析

当内核通过 netlink 从用户空间接收到规则数据时，需要转化为内核规则数据结构。下面给出了内核中规则数据结构的定义：

```
struct proto_match {  
    char left; /* 协议域匹配范围左边界 */  
    char right; /* 协议域匹配范围右边界 */  
};
```

```

struct port_match {
    unsigned short left; /* 端口域匹配范围左边界 */
    unsigned short right; /* 端口域匹配范围右边界 */
};

struct ipv6_addr_match {
    unsigned int left[4]; /* IP 地址域匹配范围左边界 */
    unsigned int right[4]; /* IP 地址域匹配范围右边界 */
    char prefix_length; /* IP 地址域前缀长度 */
};

struct hspca_rule {
    proto_match protocol; /* 规则协议域匹配范围 */
    port_match src_port; /* 规则源端口域匹配范围 */
    port_match dst_port; /* 规则目的端口匹配范围 */
    ipv6_addr_match src_ipv6_addr; /* 规则源 IP 地址匹配范围 */
    ipv6_addr_match dst_ipv6_addr; /* 规则目的 IP 地址匹配范围 */
    char action; /* 规则行为 */
    struct hspca_rule *next; /* 指向下一个规则的指针 */
};

```

规则的初始化、规则的插入、规则的删除都与相应的二叉树的操作有关，有关二叉树数据结构的定义如下所示：

```

/* 二叉树叶子节点锁包含的规则信息数据结构 */
struct information {
    int rule_num; /* 二叉树的叶子节点所包含的规则个数 */
    struct hspca_rule *pointers_to_rules[MAX_NUMBER_RULE]; /* 二叉树叶子节点中指向规则的指针数组 */
};

/* 二叉树节点的通用数据结构 */
struct binary_tree_node_spec {
    struct binary_tree_node_spec *left; /* 二叉树节点指向左子节点的指针 */
    struct binary_tree_node_spec *right; /* 二叉树节点指向右子节点的指针 */
    int position; /* 当节点为中间节点时，position 表示的是在该节点抽取的比特位的位置，需要计算得出 */
    /* 如果该节点为中间节点，则 option 指针为空，如果是叶子节点，则该指针指向叶子节点所包含的规则信息结构 */
    struct information *option;
};

/* 指向二叉树根节点的指针 */
struct binary_tree_spec {
    struct binary_tree_node_spec *head;
};

```

4.2.3.2 规则的初始化

首先把整个规则集分成 20 个规则子集。前面提到过，我们将根据规则的协议域和目的端口域把整个规则集分成 20 个规则子集。对每个规则子集构建一个二叉查找树。关键代码如下：

```
/* 定义一个指向规则指针的指针数组，长度为 20 */
struct hspca_rule **classification_pointers[20];
/* 初始化数组，使得每个指针指向的数据区可以容纳
   MAX_NUMBER_RULE_PER_FIELD 个规则指针 */
for (i = 0; i < 20; i++)
    classification_pointers[i] = (struct hspca_rule**)
        kmalloc(MAX_NUMBER_RULE_PER_FIELD
            *sizeof(struct hspca_rule *), GFP_KERNEL);
do {
    /* 判断规则属于哪个域，并把指向该规则的指针放到相应的域中 */
    rule_field = classification_rule_according_to_protocol_and_dstport
        (tmp, classification_pointers, num);
    /* 当某一个域中存放的规则已经超过预分配的规则数时，扩大该域可以存放的规
       则数(这里把容量增加 50) */
    for (i = 0; i < rule_field.number; i++) {
        /* 由于一个规则可能属于多个域，逐个域进行判断 */
        /* 如果该域存储的规则已满时，重新分配存储空间 */
        if (num[rule_field.field[i]] == max_rule_number_pointer[rule_field.field[i]] {
            /* 保存原来存放规则内存的起始地址 */
            tmp_2 = classification_pointers[rule_field.field[i]];
            /* 新分配内存空间 */
            classification_pointers[rule_field.field[i]] = (struct hspca_rule **)kmalloc
                ((max_rule_number_pointer[rule_field.field[i]] + 50)*sizeof(struct
                    hspca_rule *), GFP_KERNEL);
            /* 把原来的规则存放到新分配的空间 */
            for (j = 0; j < max_rule_number_pointer[rule_field.field[i]]; j++)
                *(classification_pointers[rule_field.field[i]]+j) = *(tmp_2+j);
            /* 释放原来的存储空间 */
            kfree(tmp_2);
            /* 改变该域最大可以存储的规则数 */
            max_rule_number_pointer[rule_field.field[i]] += 50;
        }
    }
    /* 将下一条规则进行分类 */
    tmp = tmp->next;
} while (tmp->next != NULL); /* 知道整个规则集分类结束 */
```

然后对于每个规则子集构建二叉查找树。二叉树的构建是建立在源 IPv6 地址

和目的 IPv6 地址进行位选取的基础上的。通过对源 IP 地址和目的 IP 地址的相应位进行抽取，把每个规则子集分成一个个规则数不大于 N(程序中定义为 8)的小集合存放在二叉树的叶子节点中。其关键代码如下：

```
/* 将 20 个规则子集分别构建二叉树 */
for (i = 0; i < 20; i++) {
    /* 如果要插入的规则数小于或等于叶子节点可以存储的规则数时 */
    if (num[i] <= MAX_NUMBER_RULE) {
        /* 分配指向二叉树根节点的指针 */
        trees[i].head = (struct binary_tree_node_spec *)kmalloc
            (sizeof(struct binary_tree_node_spec), GFP_KERNEL);
        /* 生成一个叶子节点 */
        initialize_new_leaf_node(trees[i].head);
        /* 利用规则填充该叶子节点 */
        trees[i].head->option->rule_num = num[i];
        for (j = 0; j < num[i]; j++)
            trees[i].head->option->pointers_to_rules[j] = *(classification_pointers[i]+j);
    } else { /* 如果该域的规则数大于叶子节点可以存储的规则数时 */
        /* 分配一个指向二叉树根节点的头指针 */
        trees[i].head = (struct binary_tree_node_spec *)kmalloc
            (sizeof(struct binary_tree_node_spec), GFP_KERNEL);
        /* 初始化一个中间节点 */
        initialize_new_middle_node(trees[i].head);
        /* 根据该域中所有的规则创建一个二叉树 */
        create_binary_tree_according_to_rules(classification_pointers[i],
            trees[i].head, num[i]);
    }
}
```

到这里，规则的初始化工作完成，规则的初始化主要完成了对根据现有的规则进行分类并且对各个规则子集构建二叉查找树。

4.2.3.3 规则的插入

由于 HiPAC 算法支持规则的动态插入和删除，所以推广到 HiPACv6 算法同样要支持规则的动态插入和删除。规则的插入操作包含以下几个步骤：

- (1) 首先将规则加入到规则链表中；
- (2) 根据规则的协议域和目的端口域判断规则需要添加哪些些二叉树中；
- (3) 把要插入的规则添加到二叉查找树种。

4.2.3.4 规则的删除

规则的删除操作分为单个规则的删除操作和整个规则的清除操作，对于整个

规则的清楚操作实现比较简单，主要包含两个步骤：

- (1) 删除规则链中所有的规则，并且释放相应的内存空间；
- (2) 删除各个规则子集构建的二叉树数据结构，同时释放相应的内存空间。

除了整个规则的删除操作之外，还可以进行动态的删除单个规则，主要实现步骤如下：

- (1) 根据输入参数的规则号，寻找要删除的规则；
- (2) 计算规则已经插入到那些二叉树中；
- (3) 从计算的各个二叉查找树中删除叶子节点数据区指向该规则的指针；
- (4) 最后从规则链中删除该规则。

4.2.3.5 报文的规则匹配

利用该算法对包进行处理首先要定义一个 hook，然后把该 hook 注册到 netfilter 上。相关代码如下：

```
/* hook 的定义如下 */
struct nf_hook_ops ipv6_hspca_option = {
    .hook    = hspca_input_match, /* hook 处理函数，当包经过该钩子点遍历到该
                                   hook 时执行 */

    .owner = THIS_MODULE,
    .pf = PF_INET6, /* 注册的协议簇为 IPv6 协议簇 */
    .hooknum = NF_IP6_LOCAL_IN, /* 注册的钩子点 */
    .priority = NF_IP6_PRI_FILTER - 1, /* 该 hook 的优先级，优先级高的 hook 先遍
                                         历 */
};

int ret;
/* 注册该 hook 到钩子点上 */
ret = nf_register_hook(&ipv6_hspca_option);
```

包在该 hook 上的处理流程如下：

- (1) 首先获取到该报文的 IP 中的协议号、目的端口号、源 IPv6 地址、目的 IPv6 地址；
- (2) 根据报文的协议值和目的端口号判断报文属于哪个规则子集；
- (3) 找到该规则子集对应的二叉树，并且把源 IPv6 地址和目的 IPv6 地址拼接起来进行二叉树的查找；
- (4) 经过前面的步骤后找到最终的叶子节点，再线性匹配叶子节点中的规则，找到最终的匹配规则。

关键代码如下：

```

/* 根据报文的协议值和目标端口号计算报文属于哪个域 */
if (tmp.protocol == 6 || tmp.protocol == 17)
    /* 当上层协议时 TCP 或者是 UDP 时才有目的端口 */
    field = compute_field_according_to_protocol_and_dstport
        (tmp.protocol, tmp.dst_port);
else if (tmp.protocol == 58) /* 如果上层协议是 ICMPv6 时, 则测规则属于第 18 个域 */
    field = 18;
else /* 如果上层协议时其他值时, 则规则术语第 19 个域 */
    field = 19;
/* 根据找到的域, 找到这个规则子集所构建的二叉树, 并且查找二叉树最终匹配的叶子节点 */
pointer_to_leaf_node = find_leaf_node_match_packet(trees[field], ip_match_field);
if (pointer_to_leaf_node == NULL) /* 表示没有与报文相匹配的规则 */
    return 0;
/* 如果找到最终的叶子节点, 则线性匹配叶子节点中存储的规则 */
number = packet_match(tmp, pointer_to_leaf_node); /* 返回与报文相匹配的规则,
                                                    number 表示第几条 */

```

其中二叉树查找的关键代码如下:

```

/* 二叉树查找关键代码 */
while (header->left != NULL) {
    unsigned short tmp;
    if (header->position < 16)
        tmp = ip_match_data.src_ip_match_field & (0x8000 >> header->position);
    else
        tmp = ip_match_data.dst_ip_match_field & (0x8000 >> (header->position - 16));
    if (tmp == 0)
        header = header->left;
    else
        header = header->right;
}

```

4.2.4 内核接收命令接口

内核通过创建 netlink 接收函数来接收用户态传给内核的命令数据。关键代码如下:

```

/* 定义一个 sock 结构体, 类似于用户态的 socket 结构体 */
static struct sock *hspca_sock = NULL;
/* 注册一个 netlink socket 接收函数, 接收函数为 hspca_data_ready
 * 内核与用户态通信使用的协议号是 NLHSPCA_PROTO, 这里定义为 28 */
hspca_sock = netlink_kernel_create(NLHSPCA_PROTO, 0, hspca_data_ready,
THIS_MODULE);

```

需要说明的是，netlink 的接收不是并不是通过内核线程来完成的，而是通过软中断调用事先指定的接收函数。代码中定义的接收函数是 hspca_data_ready。

4.3 关键技术及难点分析

4.3.1 HiPACv6 对 UDP 分片包的处理

我们知道当传输层利用 UDP 发送大包时，报文会在 IP 层被分片，使得每个包的长度不会大于 MTU。在接收端，如果注册在 NF_IP_PRE_ROUTING 钩子点上的 HiPACv6 防火墙 hook 定义的优先级高于 Linux 内核连接跟踪模块定义的优先级时，接收端将接收到分片报文。这样当 HiPACv6 防火墙提取报文源端口和目的端口时就会发生错误，因为 UDP 分片报文的 UDP 头只在第一个分片包才有，从而导致 HiPACv6 防火墙对其他 UDP 分片包的不正确处理。我们采用以下解决方法：

把 HiPACv6 防火墙 hook 的优先级定义的比连接跟踪模块优先级低，这样 HiPACv6 hook 函数对包的处理在连接跟踪模块之后。由于连接跟踪模块 hook 处理函数会对 UDP 分片报文进行重组，所以 HiPACv6 hook 函数处理的才是重组后的报文。

由于 TCP 在传输层完成分片的过程，IP 层接收到的 TCP 包都是带有 TCP 头的。所以 TCP 不会存在以上的问题。

4.3.2 HiPACv6 对加密报文的处理

我们知道 IPv4 头最短为 20 个字节，最长为 60 个字节，这是通过添加 IPv4 选项实现的。然而 IPv6 使用扩展头来取代这一方案，IPv6 头结构如图 4.1 所示：

版本	业务流类别	流标签		
净荷长度		下一个头		跳极限
源IP地址				
目的IP地址				

图 4.2 IPv6 头结构

其中下一个头字段的可能值如表 4.1 所示，该值用以指明有无扩展头，如果有，指明是哪一种扩展头。由此可知，我们无法直接从 IPv6 头中提取上层协议号，而是需要通过若干个 IPv6 扩展头的遍历，确定传输层协议号。同样，端口号也需要通过找到传输层头来获取。

然而，当 IPv6 报文某个扩展头是 ESP 头时，表示后面的数据是加密数据，不能直接获取 IPv6 报文的协议号和端口号。我们应该首先解密后面的数据，再获取

上层协议号和端口号。

表 4.1 IPv6 扩展头

下一个头字段值	描述
0	逐跳头
43	选路头(RH)
44	分段头(FH)
51	身份验证头(AH)
52	封装安全性净荷(ESP)
59	没有下一个头
60	目的地选项头

对于加密报文，解决方法如下：

查找 IPv6 扩展头时，首先判断该扩展头是否为 ESP 头，如果是，我们调用 IPsec ESP6 协议接收处理函数对包进行解密处理，然后再提取解密报文的协议号和端口号。其中 ESP6 协议接收处理函数为 xfrm6_rcv。

4.3.3 位选取法难点分析

由前面可知我们进行比特位选取时候，选取的是源 IP 地址的 16~31 位和目的 IP 地址的 16~31 位。但是当规则中 IP 地址网络号长度小于 32 位时，例如 IP 地址网络号长度为 16bit，此时我们就不能简单的认为 16~31 之间的比特位是 0 或者是 1。我们采用下面的解决方法。

当一个 IP 地址(源 IP 地址或者是目的 IP 地址)网络号长度小于 32 的规则插入规则集中时，首先根据规则的协议域范围和目的端口范围判断规则落在哪个规则子集中。然后把源 IP 地址的第 16~31 位和目的 IP 地址的 16~31 位串接，并且记录源 IP 地址的网络号长度和目的 IP 地址的网络号长度。通过查找该规则子集对应的二叉树，找到最终的叶子节点。查找过程如下：

当中间节点存储的比特位位置(源 IP 地址位置或者是目的 IP 地址位置)大于规则中与之相对应 IP 地址的网络号长度时，我们就认为该规则既属于左子节点，也属于右子节点。所以查找过程就分为两个分支，包括该中间节点的左子节点查找和右子节点查找。

当找到最终的叶子节点，并且要插入到叶子节点时，如果叶子节点等于最大允许的规则数，则需要把规则重新分配到两个叶子节点中。对于 IP 地址长度小于 32 的规则来说，我们采用以下的方法：

假设规则中源 IP 地址网络号长度为 $n < 32$ ，当我们在进行位选取时，如果计算 n 后面比特位的 $|ni1 - ni0|$ (该比特位是 1 的规则数与 0 的规则树差的绝对值)，则该

规则不列入计算的规则集中。当位选取好之后，把叶子节点中的规则和药插入的规则分成两部分时，我们首先判断规则对应 IP 地址网络号长度是否大于位选取的比特位的位置，如果不是，则该规则需要同时插入到两个叶子节点中。

4.4 本章总结

本章主要讲述了 HiPACv6 算法在 IPv6 防火墙原型系统中的实现技术，主要实现细节包括：算法在内核的操作（如规则的初始化、规则的插入、规则的删除、包的匹配、内核接收用户数据等）、用户态的操作（如命令的分析、与内核的规则发送接口等）。重点分析了 HiPACv6 对 UDP 分片报文的处理、HiPACv6 对加密报文的处理、规则 IP 地址网络号长度小于 32 时(位选取分别在源 IPv6 地址和目的 IPv6 地址的 16~31 位之间进行选择，所以最大值为 32)位选取法处理难点三个关键技术，分别提出了解决方法。

第五章 系统测试及结果分析

本章主要对实现的 IPv6 防火墙原型系统进行测试。测试包括防火墙功能测试和性能测试。首先生成 IPv6 规则集和测试包对 HiPACv6 算法进行功能测试，然后再用 iperf 工具分别对 HiPACv6 算法和 Ip6tables 进行性能测试。最后，对 HiPACv6 算法和传统的 Ip6tables 算法测试的性能数据进行分析。

5.1 IPv6 防火墙原型系统配置

IPv6 防火墙原型系统测试环境如图 5.1 所示，包含一台发包主机和一台防火墙测试主机。

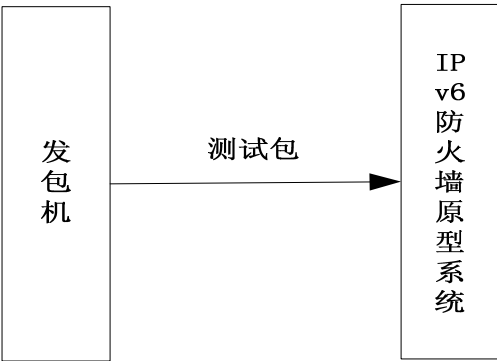


图 5.1 IPv6 防火墙原型系统测试环境

发包主机 IPv6 防火墙原型系统的软硬件配置如表 5.1 所示：

表 5.1 原型系统配置

配置 主机	CPU	内存	网卡类型	操作系统	内核版本
发包主机	IntelCore i5 双核 2.80GHz	3.21GB	千兆网卡	Redhat Linux	Linux-2.6.18
IPv6 防火 墙原型系 统	IntelCore i5 双核 2.80GHz	3.21GB	千兆网卡	Redhat Linux	Linux-2.6.18

5.2 测试方案设计

防火墙的测试首先要准备好用于测试的规则集和测试包，这里采用论文 ClassBench: A Packet Classification BenchMark^[34]中规则 and 测试包生成的方法。论文

总主要通过对路由器、ISP 防火墙规则集等进行分析,得到了企业级防火墙或者是核心路由器中源端口号、目的端口号、协议域、源 IP 地址、目的 IP 地址的分布规律。

由于该论文是针对 IPv4 下生成规则和测试包的,所以我们制定了一个生成规则和测试包的 IPv6 地址方法。由于 IPv6 地址主要包含以下几个特征:

- (1) 以 2001 为首的 IP 地址段(约占 70%); 以 2002 为首的 IP 地址段(约占 20%); 其他 IP 地址段(约占 10%);
- (2) 网络号长度分布比较集中, 长度为 24、28、32、48、64 的规则占规则总数的 90%, 其中网络号长度为 32 位占最多。约占总数的 50%^[35]。

随着 IPv6 的逐渐商业化, 这些特征或多或少会有所变化, 有待后来感兴趣的研究人员进一步研究。为了方便起见, 我们只分析源 IP 地址范围的生成, 目的 IP 地址范围的生成同理。假设我们要生成的规则数是 n :

- (1) 首先全部规则 IP 地址的前 16bit, 按照这样的比例生成: 2001 占 70%, 2002 占 20%, 其他的随机生成, 占 10%;
- (2) 然后分别随机的生成后面 112bit 数据;
- (3) 在区间[0,8]之间循环生成随机数, 循环的次数为总规则数的个数, 如果生成的随机数为(0,1,2,3,4)中的任意一个, 则该规则网络号长度取 32, 如果生成的是随机数为 5, 该规则的网络号长度取 24, 如果生成的随机数为 6, 该规则的网络号长度取 28, 如果生成的随机数为 7, 则该规则的网络号长度取 48, 如果生成的随机数为 8, 则该规则的网络号长度取 64。

通过上面的步骤, 我们就可以生成 n 个能用掩码表示的 IPv6 地址段。

5.3 防火墙算法的功能测试

生成防火墙规则集和测试包之后, 首先要对改进后的 Linux 防火墙进行功能性测试, 即验证算法的正确性。测试过程如下:

- (1) 编写 shell 脚本, 主要用于生成规则集, 并且将生成的规则集发送到内核中。启动 ip6tables 防火墙;
- (2) 用另外一台主机向 IPv6 防火墙原型系统发送测试包;
- (3) IPv6 防火墙原型系统记录防火墙对测试包的处理;
- (4) 关闭 ip6tables 防火墙, 启动本算法模块, 用同样的方法发包测试, 并记录测试结果;
- (5) 编写一个程序, 判断测试包与测试规则的匹配信息, 并且打印相关信息;
- (6) 如果三种测试结果一致, 则说明算法实现的正确性。

5.4 性能分析

分别对基于 Netfilter/Ip6tables 的防火墙和 HiPACv6 算法进行性能测试，我们用 iperf^[36]工具来分别测试两种情况下的吞吐量(单位 Mb/s)。测试结果如表 5.2 所示。

表 5.2 Ip6tables 与 HiPACv6 防火墙吞吐率的测试结果(单位 Mb/s)

规则数 算法	25	50	100	200	400	800	1600	3200	6400	12800
Ip6tables	936.2	932.3	917.4	903.7	824.3	713.7	571.6	395.6	214.8	32.9
HiPACv6	935.1	932.1	930.5	930.1	924.8	920.3	913.1	875.2	834.6	786.2

根据测试结果可知，当规则数较小时，Ip6tables 和 HiPACv6 性能相当。随着规则数的逐渐增大 Ip6tables 的性能急剧下降，而 HiPACv6 算法下降到一定程度后，趋于平稳，如图 5.2 所示。改进后的防火墙已经可以适用于企业级 IPv6 防火墙。

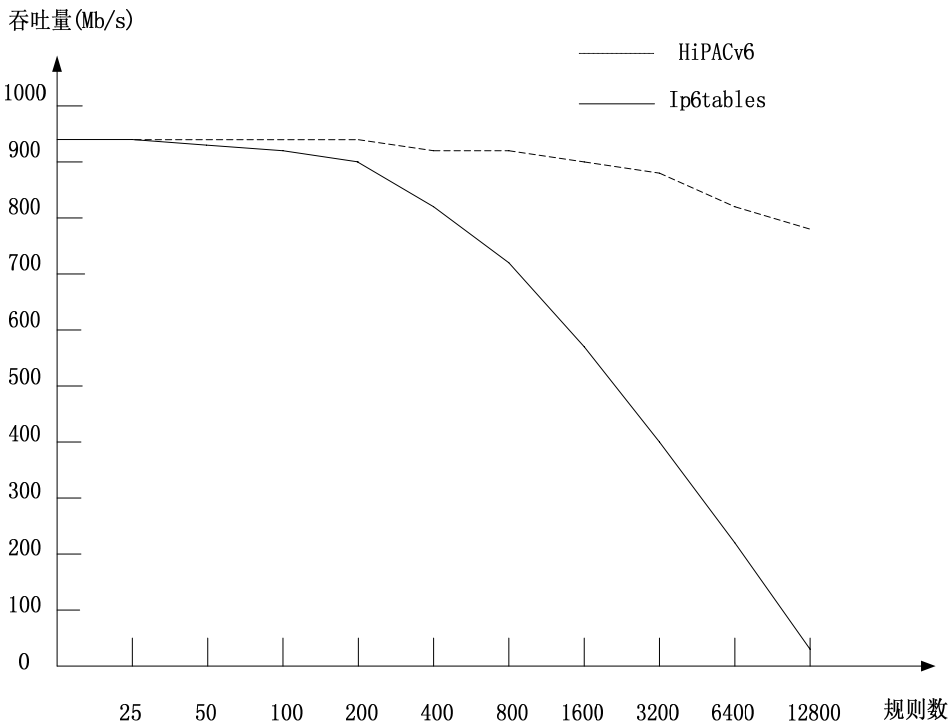


图 5.2 HiPACv6 与 Ip6tables 性能比较

5.5 本章总结

本章首先提出 IPv6 防火墙原型系统的测试方案，然后再对原型系统进行功能和性能测试，通过分析测试结果，得出 HiPACv6 算法性能大大优于 Ip6tables。

第六章 总结与展望

6.1 总结

本文以在 Linux 内核中实现 IPv6 防火墙为目标, 首先分析 Linux 下包的接收和发送流程, 并列出了相关的函数。进而引出 Linux 系统中基于 Netfilter/Iptables 的防火墙, 通过分析 Netfilter 和 Iptables 框架以及相关的函数和数据结构, 使得我们对 Linux 防火墙有一个大概的了解。然后针对基于 iptables 的防火墙不足之处, 分析了基于 IPv4 的高性能规则匹配算法 HiPAC。针对 HiPAC 算法的内存空间消耗太大的问题, 以及 IPv6 地址特征, 我们提出了基于 HiPACv6 的 IPv6 高性能规则匹配算法(因为在 IPv4 即将向 IPv6 过渡以及该算法在 IPv6 下优势比 IPv4 的更明显)。在提出 HiPACv6 算法之后, 我们设计和实现了基于 Linux 的 IPv6 防火墙原型系统, 并对其关键实现技术进行了阐述, 最后通过构造系统测试环境对 IPv6 防火墙原型系统进行了功能和性能测试。

本文包含两个重点内容: 第一, 通过对 HiPAC 的分析, 发现它的不足之处, 然后对其进行改进, 提出了适用于 IPv6 下防火墙高性能规则匹配算法; 第二, 在 Linux 内核下实现了该算法。

在 Linux 内核实现该算法是一个复杂的过程, 一开始由于对 Linux 内核不是很熟悉, 花了好长的时间看 Linux 操作系统和 TCP/IP 协议栈相关的书籍。当有了一个大概的了解之后, 就开始在 Linux 内核下完成系统的实现。在进行 Linux 内核编程时也遇到很多的困难, 但是通过反复的看书和反复的调测试实验终于解决了相关的问题。

通过对算法的改进与实现, 与基于 Netfilter/Iptables 防火墙相比, 性能有了极大的提高。测试结果表明, 对于企业级防火墙而言, 已经可以满足需求。

6.2 展望

虽然改进后的防火墙性能有了极大的提高, 但是也有不足之处, 由于课题的时间和本人的精力所限, 一些相关的工作还没有来得及做。改进的工作主要包括算法和系统实现两个方面:

第一, IPv6 引入了流标签的概念。流标签是一个在 IPv6 包头中的 20 比特字段, 主要设计目的是与源地址、目的地址一起进行流标识, 以此取代 IPv4 传统的五元组标识方式。流标签字段主要有以下几个特点:

(1) 流标签在 IPv6 固定包头中占 20 比特, 源端用其作为一个流的标签;

-
- (2) 流标签值为 0 标识该数据包不属于任何流;
 - (3) 非零流标签值, 源地址, 目的地址可在全球范围内唯一标识一个流;
 - (4) 建立流状态信息的节点对数据包进行逐流处理;
 - (5) 流标签值自源端到目的端全程不变;
 - (6) 限制生存时长为 120 秒。

所以基于流标签规则匹配可以将原来 IPv4 下的五元组匹配转变 IPv6 三元组匹配, 即基于源 IP 地址、目的 IP 地址、流标签的三元组匹配。利用 IPv6 的这个特点可以大大提高规则匹配的性能。

第二, 首先该算法实现除了支持五元组匹配之外, 暂时还不支持其它域的匹配。防火墙的匹配域包括很多方面, 最基本的匹配域就是五元组匹配(源 IP 地址、目的 IP 地址、源端口、目的端口、协议号), 如果需要在算法中增加其他的匹配域, 其中一种最简单的方法就是先把报文的五元组提取出来, 分类查找二叉树, 首先对五元组进行匹配, 看包被匹配到了哪个叶子节点, 在分别对叶子节点中的规则进行完全匹配。当然还有其他更好的方法。其次, 该算法只支持用掩码形式表示的 IPv6 地址范围的规则。再次, 该算法在用户态下规则的配置方式都是通过命令行形式(shell)输入的。对于一般的用户来说使用非常不方便, 同时使得过滤规则的扩充和修改也极为繁琐。为了更好的有效匹配 IPv6 防火墙过滤规则, 可以设计一个配置过滤规则的图形界面, 从而可以方便快捷地应用防火墙。由于时间所限, 本文中没有添加此功能。

总之, 防火墙作为网络安全的卫士, 在网络飞速发展的今天, 发挥着越来越重要的作用。在网络流量日益增加的情况下, 对防火墙处理效率的要求也日益增强。Linux 系统作为一种开源的操作系统, 已经被越来越多的个人、企业和机构组织采用。因此, 研究 Linux 防火墙, 尤其是 IPv6 下的 Linux 防火墙有着十分重要的意义。本文正是从这些实际需求出发, 对 IPv6 下 Linux 防火墙作了较大改进, 从而大大提高了防火墙的效率。

致 谢

在我的课题和硕士论文完成之际，谨向在我攻读硕士学位过程中曾经指导过我的老师，关心过我的朋友，关怀过我的领导，以及所有帮助过我的人们致以崇高的敬意和深深的感谢！

衷心感谢我的导师王晓东和指导导师王勇军。两位老师平时研究工作非常繁忙，但仍在百忙之中抽出时间对我进行全面的指导。感谢他们给予我的悉心指导和热情帮助。无论是在论文的选题、章节的具体研究内容、还是在论文的最后定稿等方面都倾注了两位老师的大量心血；在论文的写作期间，论文的每一稿他们都仔细审阅，并给予详细的批注指导，指导和协助我解决遇到的实际问题。在这里，我不仅学到了专业知识，治学严谨、工作认真负责的精神也深深地影响了我。我对两位老师的感激之情，难以言表，惟借此文向他们致以最诚挚的谢意！

感谢唐勇、庞立会老师。老师们知识渊博、工作严谨求实、思维开阔、工作态度积极，在本论文的实现过程和论文写作方面，使我获益匪浅。

感谢我的师兄解培岱、李美健、周东浩、赖志权，他们无论是在学习上，还是在生活上都给予了我坚定的支持。同时感谢软件工程硕士队的李元、张潇晓、纪占龙等同学，在做课题的那段艰苦的时间里大家一起工作、学习，既有技术上的支持，又感受到了大家庭的温暖。感谢罗秀昌、韩子龙、尤作赛、刘星等师弟，在一起的岁月中给我带来的快乐。

特别感谢我的父母在生活中给予的关怀，他们对我的鼓励是促使我不断进取的巨大精神动力！无论成功和失败，他们都是我最忠实的支持者。

感谢我周围的朋友，在我进行大论文书写时，对我精神上的鼓励，使我勇敢的面对了很多的困难，真正的感受到了友谊的可贵之处。最后，再次向所有在我攻读硕士学位的过程中曾经指导我的老师，关怀我的领导，关心和帮助过我的朋友致以真挚的谢意！

参考文献

- [1] Steve Suering, Robert L. Ziegler. Linux.防火墙[M].何涇沙译.北京:机械工业出版社,2006:15,42~45.
- [2] Pete Loshin. IPv6 详解(沙裴,程立,周立)[M].北京:机械工业出版社,2000:43~78.
- [3] Gupta P., McKeown N. Algorithms for Packet Classification[J]. IEEE Network, 2001, 15(2): 24~32.
- [4] 宋舜宏.大规则集下 Linux 防火墙的优化[D].合肥:中国科技大学.2005:51.
- [5] Bellion M. Optimizing HiPAC for policy routing, packet filtering and traffic
- [6] Pankaj Gupta, Nick McKeown. Classifying Packet with Hierarchical intelligent cuttings[J]. Proc.ACM SIGCOMM, 2000, 15[2]:21~34.
- [7] Taylor D. E. Survey and Taxonomy of Packet Classification Techniques[J]. ACM Computing Surveys, 2005, 37(3): 238~275.
- [8] Bradner S. Benchmarking terminology for network inter-connection device. IETF(RFC 1242), 1999-7.
- [9] Ramaswamy Ramaswamy, Ning Weng, Tilman Wolf. Analysis of network processing workloads[J]. Journal of Systems Architecture: the EUROMICRO Journal, 2009.
- [10]Feldmann A., Muthukrishnan S. Tradeoff for packet classification[C]. In proc of INFOCOM. Heidelberg: Springer, 2002, 1193~1202.
- [11]唐文. Netfilter 实现机制分析[EB/OL]. <http://linux.chinaunix.net/bbs/viewthread.php?tid=1054981>. 2008-12.
- [12]董良杰. Linux 防火墙介绍[J]. 科学教育专家,2008, 9(上):22~24.
- [13]倪继利 .Linux 内核分析及编程 [M]. 北京:电子工业出版社, 2005:660~663,490~494.
- [14]Christian Benvenuti.Understanding Linux Network Internals[M].United States of America:O'Reilly Media,2006:45~79.
- [15]Robert Love.Linux Kernel Development[M].United States of America:O'Reilly Media,2006:79~107.
- [16]Greg Holden.防火墙与网络安全(入侵检测与 VPNs) [M].王斌,孔璐译.清华大学出版社,2004:10~25.
- [17]雷渊明.基于 Netfilter 的包分类研究与设计[D].长沙:湖南大学, 2009: 25~49.
- [18]陈鑫 .Linux 系统内核空间与用户空间通信的实现与分析 [EB/OL]. www.ibm.com/developerworks/cn/linux/l-netlink/index.html,2004-10-24.
- [19]杨沙洲 . Linux Netfilter 实现机制和扩展技术 [EB/OL].www.ibm.com/developerworks/cu/linux/l-ntflt/index.html, 2003-4-15.

-
- [20]朱立才,杨寿保,宋舜宏.Netfilter/IPtables 防火墙性能优化与实现[J].计算机科学与应用,2006,15(42):117~120.
- [21]孙毅,刘彤,蔡一兵等.报文分类算法研究[J].计算机应用研究,2007,24(4):5~11
- [22]Mark Allen Weiss.Data Structures and Algorithm Analysis in C++[M].United States of America:Pearson Education,2006:176~193.
- [23]Brownlee N., Claffy K. Understanding Internet Traffic Stream: Dragonies and Tortoises[J]. IEEE Communications Magazine, 2002,40(10):110~117.
- [24]Gajanan S. Jedhe, Arun Ramamoorthy, Kuruvilla Varghese. A Scalable High Throughput Firewall in FPGA[C]. Proceedings of the 2008 16th International Symposium on Field-Programmable Custom Computing Machines, 2008.
- [25]Xin Li, Zheng-Zhou Ji, Ming-Zeng Hu. Stateful Inspection Firewall Session Table Processing[C]. Proceedings of the International Conference on Information Technology: Coding and Computing (ITCC'05) - Volume II, 2005.
- [26]Kristen Accardi, Tony Bock, PFrank Hady, Jon Krueger. Network processor acceleration for a Linux netfilter firewall[C]. Proceedings of the 2005 ACM symposium on Architecture for networking and communications systems, 2005.
- [27]Daniel P.Bovet,Marco Cesati.Understanding the Linux Kernel[M].United States of America:O'Reilly Media,2006:276~293.
- [28]He K. Why and How to Use Netlink Socket. Linux Journal, 2005,2005(130):11.
- [29]Janardhan Singaraju, John A. Chandy. FPGA based string matching for network processing applications[J]. Microprocessors & Microsystems, 2008.
- [30]史蒂文斯.UNIX 环境高级编程[M].北京:机械工业出版社,2006:25~450.
- [31]毛德操,胡希明.Linux 内核源代码情景分析.浙江:浙江大学出版社,2001:45~69
- [32]李善平,陈文智.边学边干-Linux 内核指导[M].浙江:浙江大学出版社,2005:98~127.
- [33]范磊.Linux 内核源代码[M].北京:人民邮电出版社,2002:243~262.
- [34]David E. Taylor,Jonathan S. Turner, "ClassBench:A Packet Classification Bechmak," WUCSE-2004-28,May 21,2004.
- [35]Bradner S., McQuaid J. Benchmarking methodology for network inter-connection devices. IETF(RFC 2544), 1999-3.
- [36]Zhang Y., Breslau L.,Paxson V. et al. On the Characteristics and Origins of Internet Flow Rates[C]. In proc. of Sigcomm 2002, USA, 2002, 32(4):309~322.
-

作者在学习期间取得的学术成果

- [1] 江峰,王勇军,王晓东,解培岱.IPv6防火墙高性能规则匹配算法研究.2011网路与信息安全会议. (已录用).